

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

MASTER THESIS

Bc. Norbert Michel

Min Cut-Path

Department of Applied Mathematics

Supervisor of the master thesis: prof. RNDr. Martin Loebl, CSc.

Study programme: Discrete Models and Algorithms

Prague 2025

I declare that I carried out this master thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

My sincere gratitude goes to my supervisor for suggesting such an original and thought-provoking thesis topic, which I believe holds potential for further research. I also would like to thank everyone who offered valuable advice and guidance to improve my work.

Title: Min Cut-Path

Author: Bc. Norbert Michel

Department: Department of Applied Mathematics

Supervisor: prof. RNDr. Martin Loebl, CSc., Department of Applied Mathematics

Abstract: This thesis addresses the Min Cut-Path problem: finding a minimal set of edges that simultaneously contains both a u - v cut and a u - v path in a graph. The combination of these two concepts increases the complexity of the problem beyond that of standard algorithms for cuts or paths. We identify and analyze the conditions under which the Min Cut-Path problem allows efficient solutions. For graphs with diam or cut 2, that is, for all x, y with $d(x,y) \leq 2$ or $c(x,y) \leq 2$, we propose a polynomial-time algorithm for the case $c(u, v) = 2$, based on a decomposition using general square graphs. In Erdős–Rényi random graphs, we present an approximation scheme with an average-case guarantee arbitrarily close to 1. Furthermore, we propose an almost polynomial-time algorithm for sparse instances. Finally, we study the symmetric case $c(u,v) = d(u,v) = cp(u,v)$ and provide polynomial-time algorithms for recognizing this property and finding solutions in nearly 5-regular graphs.

Keywords: Graph, edge-cut, path, cut-path, general path, square graph, general square graph, diam or cut 2 graph, nearly k -regular graphs, symmetric cut-path

Název práce: Minimalni rez-cesta

Autor: Bc. Norbert Michel

Katedra: Katedra aplikované matematiky

Vedoucí diplomové práce: prof. RNDr. Martin Loebl, CSc., Katedra aplikované matematiky

Abstrakt: Tato diplomová práce se zabývá problémem minimalni rez-cesta: hledáním minimální množiny hran, která současně obsahuje u - v řez i u - v cestu v grafu. Spojení těchto dvou konceptů zvyšuje složitost problému nad rámec standardních algoritmů pro řez nebo cestu. Identifikujeme a analyzujeme podmínky, za kterých má problém minimalni rez-cesta efektivní řešení. Pro grafy s průměrem nebo řezem nanejvýš dva, tedy když pro všichni x, y $d(x,y) \leq 2$ nebo $c(x,y) \leq 2$, navrhujeme polynomiální algoritmus pro případ $c(u,v) = 2$, založený na dekompozici pomocí obecných čtvercových grafů. V Erdős–Rényi náhodných grafech prezentujeme aproximační schéma s průměrnou zárukou přesnosti libovolně blízké 1. Dále navrhujeme téměř polynomiální algoritmus pro řídké instance. Nakonec se zabýváme symetrickým případem $c(u,v) = d(u,v) = cp(u,v)$ a poskytujeme polynomiální algoritmy pro rozpoznání této vlastnosti a pro hledání řešení v téměř 5-regulárních grafech.

Klíčová slova: Graf, hranový řez, cesta, řezo-cesta, obecná cesta, čtvercový graf, obecný čtvercový graf, graf s průměrem nebo řezem nejvýše dva, téměř k -regulární graf, symetrická řezo-cesta

Contents

Introduction	7
1 Foundations	9
1.1 Graph	9
1.1.1 Degree	9
1.1.2 Path and Cycle	10
1.1.3 Cut	11
1.1.4 Tree and Spanning Tree	11
1.1.5 Graph Transformations	12
1.1.6 Useful Theorems	13
1.2 Algorithm	13
1.2.1 Algorithm	13
1.2.2 Approximation and Polynomial-Time Algorithm	15
1.3 Probability Theory	15
2 Fundamentals of Cut-Path Problem	18
2.1 Cut-Path	18
2.2 Tree-Cut	19
2.3 Basic Properties of Cut-Path	21
2.4 Exploiting Partial Solutions	23
3 Min Cut-Path Problem in Special Graph Classes	25
3.1 Graphs of Diameter Two	25
3.2 Graphs with Cut-Value at Most Two	31
3.3 Diam or Cut 2 Graph	33
3.3.1 General Square Graph	34
3.3.2 Transformations	37
3.3.3 Polynomial-Time Construction of General Square Graph	42
3.3.4 Polynomial-Time Algorithm for Min Cut-Path in Diam or Cut 2 Graph	45
3.3.5 Linear-Time Algorithm for Min Cut-Path with General Square Graph Decomposition	48
4 Min Cut-Path Problem in Random Graphs	52
4.1 Time Complexity Classes for Random Graph Algorithms	52
4.2 Diameter of Erdős-Rényi $G(n, 1/c)$ Graphs	54
4.3 Properties of Erdős-Rényi Random Graphs	55
4.4 Average-Case Approximation for Min Cut-Path in Erdős-Rényi Graphs	56
4.5 Almost Polynomial Average-Case Approximation Algorithm in Erdős-Rényi Graphs	57
5 Min Cut-Path Problem: The $c(u, v) = d(u, v) = cp(u, v)$ Case	60
5.1 Fundamentals of $c(u, v) = d(u, v) = cp(u, v)$ Case	60
5.2 Algorithmic Solutions for Min Cut-Path in $c(u, v) = d(u, v) = cp(u, v)$ Case	64

5.2.1	Filter-BFS Algorithm	64
5.2.2	Local-Cut Algorithm	66
5.2.3	Polynomial Cases: Nearly k-Regular Graphs	67
5.2.4	Limitations of Filter-BFS and Local-Cut Algorithms	69
Conclusion		71
Bibliography		73
List of Figures		74
List of Tables		75
List of Abbreviations		76
A Attachments		77

Introduction

Many fundamental graph problems are related to connectivity or separation between nodes, which is formally captured by the concepts of paths and cuts. This thesis investigates a novel optimization problem that integrates both: the Min Cut-Path problem.

The Min Cut-Path problem emerges from the combination of these two foundational concepts. The integration of cuts and paths is of particular interest due to the contrasting behavior of these concepts. For example, in denser graphs, minimum cut values typically increase, while shortest path distances tend to decrease. Combining these requirements into a single objective function results in a complex interplay, which poses a significant optimization challenge.

The Min Cut-Path problem is considerably more intricate than its individual components. A key difficulty arises from the fact that the union of an edge set that produces a minimum cut $c(u, v)$ and an edge set that realizes a shortest path $d(u, v)$ does *not*, in general, yields an optimal solution. Specifically, the minimum cut-path $cp(u, v)$ cannot be determined solely from $c(u, v)$ and $d(u, v)$, as the optimal solution does not necessarily correspond to their union. This observation underscores the need for new algorithmic techniques beyond standard approaches for shortest paths and minimum cuts. Despite the general case appearing to be computationally intractable, a formal proof of *NP-hardness* remains an open problem.

This thesis focuses on the structural properties and algorithmic aspects of the Min Cut-Path problem. The goal is to identify conditions and graph classes where the problem is tractable, or at least approximable, and to develop efficient algorithms for these cases. The primary challenges addressed include:

- Developing polynomial-time algorithms for Min Cut-Path on specific, structurally constrained graph classes.
- Analyze the behavior and complexity of Min Cut-Path in random graphs and investigate its average-case characteristics.
- Characterization of the special symmetric case where $c(u, v) = d(u, v) = cp(u, v)$.

This work contributes to understanding the Min Cut-Path problem by providing several novel results. In Chapter 2, we establish fundamental properties and bounds. In Chapter 3, we investigate specific graph classes, culminating in a polynomial-time (and potentially linear-time) algorithm for the important class of *diam or cut 2* graphs, using a novel decomposition technique based on *general square graphs*. Chapter 4 explores the behavior of the problem in random graphs, presenting an average-case approximation scheme for dense Erdős-Rényi random graphs and an almost polynomial-time average-case algorithm for sparse instances. Finally, Chapter 5 addresses the symmetric case $c(u, v) = d(u, v) = cp(u, v)$, providing polynomial-time algorithms to decide this property and solve instances for *nearly 5-regular* graphs.

Structure of the Thesis

The thesis is structured as follows:

- **Chapter 1: Foundations.** Introduces the fundamental concepts from graph theory, algorithms, and probability theory necessary for understanding the following chapters.
- **Chapter 2: Fundamentals of the Cut-Path Problem.** Formally defines the Min Cut-Path problem, establishes basic properties, bounds, and relationships with standard cuts and paths, and introduces the related Tree-Cut concept.
- **Chapter 3: Min Cut-Path Problem in Special Graph Classes.** Focuses on developing efficient algorithms for Min Cut-Path in graphs with specific structural properties, including graphs with diameter two, graphs with cut-value at most two, and the combined *diam or cut 3* class. Introduces the *general square* graph decomposition technique.
- **Chapter 4: Min Cut-Path Problem in Random Graphs.** Analyzes the Min Cut-Path problem within the context of *Erdős-Rényi* random graphs, examining average case complexity, approximation schemes, and developing an almost polynomial average case algorithm for sparse instances.
- **Chapter 5: Min Cut-Path Problem: The $c(u, v) = d(u, v) = cp(u, v)$ Case.** Investigates the structural implications and algorithmic solutions for the special symmetric case, including algorithms for *nearly 5-regular* graphs.
- **Conclusion.** Summarizes the main findings of the thesis and suggests directions for future research.

1 Foundations

This chapter introduces definitions and fundamental concepts essential for understanding the subsequent discussion in this thesis. We establish notation and present key theorems from graph theory, algorithms, and probability that will be necessary in the analysis in the min cut-path problem and other related topics explored in later chapters.

1.1 Graph

Graphs serve as a fundamental mathematical structure for modeling a wide range of optimization problems in computer science. By representing complex systems as graphs, one can systematically analyze network flows, connectivity properties, and structural constraints. The fundamental concepts of *graph theory*, such as *connectivity* and *cuts*, provide the theoretical basis for designing efficient algorithms for the **Minimum Cut-Path** problem. A comprehensive treatment of these *graph-theoretic* principles can be found in [1].

Definition 1 (Graph). A **graph** G is an ordered pair (V, E) , where:

- V is a set of vertices (nodes), representing the fundamental elements of the graph.
- E is a set of edges, where each edge is a pair of vertices $\{u, v\}$ for $u, v \in V$.

A graph can be directed (edges are ordered pairs) or undirected (edges are unordered pairs).

Definition 2 (Component). A **component** of a graph G is a maximal connected subgraph of G ; that is, a subgraph in which any two vertices are connected by a path and that is not contained in any larger connected subgraph.

Definition 3 (k -edge-connectivity). A graph G is **k -edge-connected** if it remains connected after removal of any set of at most $k - 1$ edges. Formally: for every set $F \subseteq E$ with $|F| < k$, the graph $G \setminus F$ is connected.

1.1.1 Degree

The *degree* of a vertex, defined as the number of incident edges, describes its local *connectivity* and structural significance in the graph. In *path* and *cut* problems, the *degree* plays a crucial role in determining feasible separations and *connectivity* constraints, directly impacting the efficiency of the algorithms for the *Minimum Cut-Path* problem.

Definition 4 (Degree Vertex). Let $G = (V, E)$ be a graph, and let $v \in V$ be a vertex. The degree of v , denoted $\deg(v)$, is the number of edges in G that contain the vertex v . For an undirected graph:

$$\deg(v) = |\{\{u, v\} \in E\}|.$$

Definition 5 (Vertex Degree With Respect to Set). Let $G = (V, E)$ be a graph, $S \subseteq V$ be a set of vertices, and $v \in V$ be a vertex. The degree of v with respect to S , denoted $\deg(v, S)$, is the number of vertices $u \in S$ such that $(u, v) \in E$. Formally:

$$\deg(v, S) = |\{u \in S \mid \{u, v\} \in E\}|.$$

Definition 6 (Isolated Vertex). Let $G = (V, E)$ be a graph. A vertex $v \in V$ is called **isolated** if its degree is $\deg(v) = 0$.

1.1.2 Path and Cycle

In this work, we define fundamental concepts such as *path* and *cycle*, which are essential for analyzing graph structures. A *path* is a sequence of distinct vertices connected by edges, while a *cycle* is a closed path where the first and last vertex coincide.

Definition 7 (Path). Let $G = (V, E)$ be a graph. A **path** in G is a sequence of vertices $P = (v_0, v_1, \dots, v_k)$, where $v_i \in V$ for all $i \in \{0, 1, \dots, k\}$ and for each $i \in \{1, 2, \dots, k\}$, $\{v_{i-1}, v_i\} \in E$. The length of the path P is the number of edges in the path, which is k .

Definition 8 (Paths). Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The set of all paths between u and v , denoted by $\text{path}(u, v)$, is defined as:

$$\text{path}(u, v) = \{P \mid P \text{ is a path between } u \text{ and } v\}.$$

Definition 9 (Cycle). A **cycle** in a graph $G = (V, E)$ is a sequence of vertices $C = (v_0, v_1, \dots, v_k)$ such that:

- $v_i \in V$ for all $i \in \{0, 1, \dots, k\}$,
- $\{v_{i-1}, v_i\} \in E$ for all $i \in \{1, 2, \dots, k\}$,
- $v_0 = v_k$,
- All edges $\{v_{i-1}, v_i\}$ are distinct, and all vertices $v_0, v_1, \dots, v_{k-1}$ are distinct.

The **length** of the cycle is k , the number of edges in C .

Definition 10 (Distance). Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The distance between u and v , denoted by $d(u, v)$, is defined as the minimum number of edges in any path between u and v . Formally:

$$d(u, v) = \min\{|P| \mid P \in \text{path}(u, v)\}.$$

In this thesis, we will refer to a *min path* (or shortest path) between the vertices u and v as any path $P \in \text{path}(u, v)$ whose length is equal to the distance $d(u, v)$, i.e., $|P| = d(u, v)$.

1.1.3 Cut

Cuts are fundamental in analyzing connectivity and are crucial in problems such as the *min cut*, where the goal is to determine optimal separation while preserving certain structural properties.

Definition 11 (Cut). *Let $G = (V, E)$ be a graph, and let $S \subseteq V$ be a subset of vertices. The cut determined by S is the set of edges $\delta(S) \subseteq E$, where each edge $e \in \delta(S)$ connects a vertex in S to a vertex in $V \setminus S$. Formally:*

$$\delta(S) = \{\{u, v\} \in E \mid u \in S, v \in V \setminus S\}.$$

Definition 12 (Cuts). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. A cut between u and v is a set of edges $C \subseteq E$ such that the removal of C disconnects u and v . The set of all cuts between u and v , denoted by $\text{cut}(u, v)$, is defined as:*

$$\text{cut}(u, v) = \{C \subseteq E \mid \text{cut separating } u \text{ from } v\}.$$

Definition 13 (Value of Minimum Cut). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The value $c(u, v)$ is defined as the minimum size of a cut between u and v . Formally:*

$$c(u, v) = \min\{|C| \mid C \in \text{cut}(u, v)\}.$$

In this thesis, we will refer to a *min cut* between the vertices u and v as any cut $C \in \text{cut}(u, v)$ whose size is equal to the minimum cut value $c(u, v)$, i.e., $|C| = c(u, v)$.

1.1.4 Tree and Spanning Tree

One of the more structured forms of graphs are *trees* and *spanning trees*, which play a crucial role in connectivity analysis. A *tree* is an acyclic connected graph, while a *spanning tree* of a graph is a subgraph that includes all vertices and forms a tree.

Definition 14 (Tree). *A graph $G = (V, E)$ is a **tree** if it satisfies the following conditions:*

- G is connected.
- G contains no cycles.

Definition 15 (Spanning Tree). *Let $G = (V, E)$ be a connected graph. A **spanning tree** of G is a subgraph $T = (V, E_T)$ such that:*

- T is a tree (i.e., it is connected and acyclic),
- $V(T) = V(G)$.

For better understanding, we provide an illustrative example:

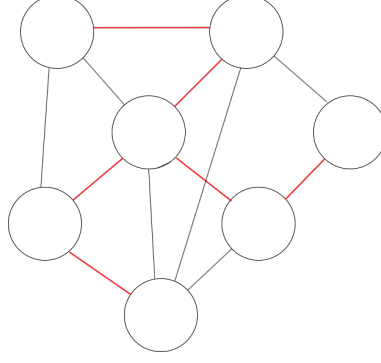


Figure 1.1 Spanning Tree

Definition 16 (Value of Minimum Tree). *Let $G = (V, E)$ be a graph. The minimum size of any spanning tree of G , denoted by $t(G)$, is defined as:*

$$t(G) = |V| - 1.$$

1.1.5 Graph Transformations

In this subsection, we define several transformations, namely *subgraph* and *contraction*, which help extract essential structural information from the original graph. A *subgraph* is a graph formed by a subset of vertices and edges of the original graph, while *contraction* reduces a graph by merging adjacent vertices, preserving key connectivity properties.

Definition 17 (Contraction of edge). *Let $G = (V, E)$ be a graph, and let $e = \{u, v\} \in E$ be an edge that connects vertices u and v . The **contraction of edge e** produces a new graph $G' = (V', E')$, where:*

$$V' = V \setminus \{u, v\} \cup \{w\},$$

$$E' = \left\{ \{w, x\} \mid \{s, x\} \in E \setminus \{e\} \wedge s \in \{u, v\} \right\} \cup \left\{ \{x, y\} \in E \mid x, y \notin \{u, v\} \right\},$$

where w is a new vertex replacing u and v .

For better understanding, we provide an illustrative example:

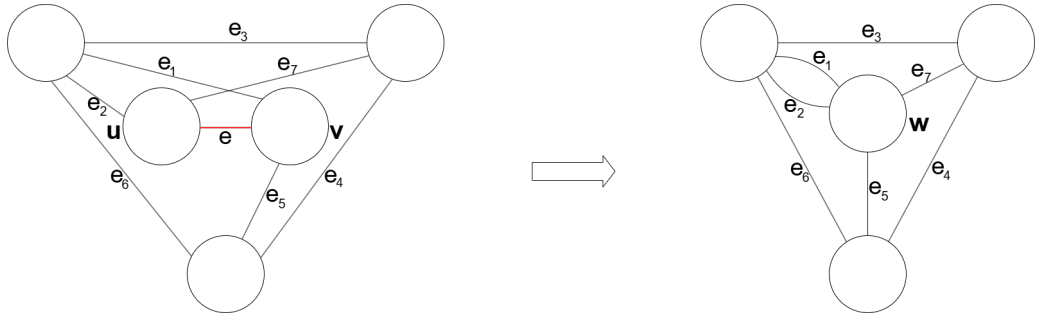


Figure 1.2 Contraction of edge e

Definition 18 (Subgraph). *Let $G = (V, E)$ be a graph. A graph $H = (V', E')$ is a **subgraph** of G if:*

- $V' \subseteq V$,
- $E' \subseteq E$.

1.1.6 Useful Theorems

In this work, we will frequently make use of Menger's Theorem, both in its edge and vertex versions. These fundamental results establish a deep connection between the concepts of cuts and paths by stating that the size of a minimum cut equals the maximum number of pairwise independent paths. This duality will serve as a key tool in the analysis of the Min Cut-Path problem.

Theorem 1 (Menger's Theorem (Vertex Version)). *Let $G = (V, E)$ be a graph, and let u and v be two non-adjacent vertices in G . Then the minimum number of vertices that must be removed from G to separate u and v into different connected components is equal to the maximum number of internal vertex-disjoint paths between u and v .*

Theorem 2 (Menger's Theorem (Edge Version)). *Let $G = (V, E)$ be a graph, and let u and v be two distinct vertices in G . Then the minimum number of edges that must be removed from G to separate u and v into different connected components is equal to the maximum number of edge-disjoint paths between u and v .*

1.2 Algorithm

This chapter provides a brief overview of essential algorithmic terminology and classifications. These foundational concepts are necessary for the algorithmic treatment of the Min Cut-Path problem in the following chapters.

1.2.1 Algorithm

An *algorithm* is a finite and well-defined sequence of computational steps that transforms input into output to solve a specific problem. Each step in an algorithm is composed of basic *operations*, which are elementary actions assumed to take constant time, such as comparisons, assignments, or arithmetic calculations. The efficiency of an algorithm is often measured by the number of operations it performs as a function of the input size. A detailed exposition of these fundamental notions can be found in [2], which also provides comprehensive introductions to essential algorithms such as *Breadth-First Search (BFS)*, *Dijkstra's algorithm* for shortest paths and *Ford-Fulkerson's algorithm* for computing maximum flow in networks.

Definition 19 (Operation). *An **operation** is a fundamental computational step that transforms a given state into a new state according to predefined rules. Formally, an operation is a mapping:*

$$\mathcal{O} : S \rightarrow S,$$

where S is the set of all possible computational states, and for each state $s \in S$, the application of $\mathcal{O}$ results in a new state $\mathcal{O}(s)$.

An operation satisfies the following properties:

- Each operation $\mathcal{O}$ is well-defined, meaning that for every input state s , there exists a unique output state $\mathcal{O}(s)$.
- A sequence of operations $(\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_k)$ can be applied iteratively, producing a sequence of states $(s_1, s_2, \dots, s_{k+1})$, where $s_{i+1} = \mathcal{O}_i(s_i)$ for each $i \in \{1, 2, \dots, k\}$.
- The execution of each operation is completed in a bounded number of computational steps.

Definition 20 (Algorithm). Let I be the set of inputs and O the set of outputs. An **algorithm** is formally defined as a computational procedure:

$$A : I \rightarrow O,$$

where for every input $x \in I$:

- There exists a finite sequence of operations $A(x) = (\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_k)$, where s_i is the computational state at step i ,
- For each $i \in \{1, 2, \dots, k-1\}$, the state s_{i+1} is uniquely determined by the rules of A from s_i ,
- The final state s_k provides the output $A(x) \in O$.

An *optimal algorithm* is one that always produces the best possible solution to a given problem according to a specified objective function. In the context of optimization problems, such an algorithm guarantees correctness and achieves the minimum (or maximum) value among all feasible solutions. Moreover, when analyzing optimal algorithms, we are often interested not only in the quality of the solution but also in their computational efficiency.

Definition 21 (Optimal Algorithm). Let $\mathcal{P}$ be an optimization problem with an objective function $f : S \rightarrow \mathbb{R}$, where S is the set of feasible solutions. An algorithm A is **optimal** for $\mathcal{P}$ if for every input $x \in I$:

$$A(x) \in \arg \min_{s \in S(x)} f(s) \quad (\text{minimization}),$$

$$A(x) \in \arg \max_{s \in S(x)} f(s) \quad (\text{maximization}),$$

where $S(x) \subseteq S$ is the set of feasible solutions for input x .

For the sake of brevity, we will denote an optimal algorithm as OPT in the subsequent discussions.

1.2.2 Approximation and Polynomial-Time Algorithm

A *polynomial-time algorithm* is an algorithm whose execution time is bounded by a polynomial function of the input size, typically denoted by $O(n^k)$ for some constant k . This means that as the input size grows, the time required to compute the result grows at a rate that is manageable and considered efficient in theoretical computer science. Formally:

Definition 22 ($O(f(n))$). Let $f : \mathbb{N} \rightarrow \mathbb{R}^+$ be a function. A function $g(n)$ is said to be **$O(f(n))$** (big O of $f(n)$) if there exist positive constants c and n_0 such that

$$0 \leq g(n) \leq c \cdot f(n) \quad \text{for all } n \geq n_0.$$

Definition 23 (Polynomial-Time Algorithm). An algorithm A is **polynomial-time** if there exists a polynomial $P(n)$ such that for every input x of length $n = |x|$, the number of operations $T_A(x)$ performed by A satisfies:

$$T_A(x) \leq P(n).$$

Equivalently, the time complexity of A is $O(P(n))$.

When no polynomial-time algorithm is known to solve a problem optimally, one often resorts to *approximation algorithms*. These algorithms do not necessarily produce the best possible solution, but guarantee that the solution is within a certain factor of the optimal one. Approximation algorithms are especially useful for tackling computationally hard problems where exact solutions are too costly to compute in practice.

Definition 24 (Approximation Algorithm). Let $\mathcal{P}$ be an optimization problem with optimal value $\text{OPT}(x)$. An algorithm A is an **approximation algorithm with approximation factor** $\alpha \geq 1$ if:

- A is a polynomial-time algorithm,
- For all $x \in I$:

$$\frac{A(x)}{\text{OPT}(x)} \leq \alpha \quad (\text{for minimization}),$$

$$\frac{\text{OPT}(x)}{A(x)} \leq \alpha \quad (\text{for maximization}).$$

1.3 Probability Theory

In this thesis, we are interested not only in the worst-case behavior of algorithms for the Min Cut-Path problem, but also in their performance on *random* inputs. To this end, we will use tools from probability theory to analyze the complexity of the average case. In our setting, we will work with finite graphs and discrete structures, so we do not need the full machinery of measure theory. Instead, we *introduce* the notion of a *finite discrete probability space*, which is a simple and intuitive model sufficient for our purposes. It allows us to reason about random choices,

expected values, and average case performance in a mathematically rigorous way, while remaining within a finite and combinatorial framework [3].

To formally describe randomness, we begin by introducing the notion of a *probability measure*, which assigns numerical values to events and underpins the entire probabilistic model. Once this measure is established, we define a *finite discrete probability space* that captures both the sample space and the structure of events.

Definition 25 (Probability Measure). *Let $\Omega = \{\omega_1, \omega_2, \dots, \omega_n\}$ be a finite sample space, and let 2^Ω be the set of all possible events (subsets of Ω). A **probability measure** $\mathbb{P}$ on $(\Omega, 2^\Omega)$ is a function $\mathbb{P} : 2^\Omega \rightarrow [0, 1]$ determined as follows:*

- $\mathbb{P}(\{\omega\}) \geq 0$ to each elementary event $\{\omega\}$ (where $\omega \in \Omega$).
- $\mathbb{P}(\Omega) = 1$.
- The probability of any event $A \in 2^\Omega$ is then defined as:

$$\mathbb{P}(A) = \sum_{\omega \in A} \mathbb{P}(\{\omega\}).$$

This leads us to the formal definition of the *finite discrete probability space*.

Definition 26 (Finite Discrete Probability Space). *A **finite discrete probability space** is a triple $(\Omega, 2^\Omega, \mathbb{P})$, where:*

- $\Omega = \{\omega_1, \omega_2, \dots, \omega_n\}$ is a non-empty, **finite** set, representing all possible outcomes (the **sample space**).
- 2^Ω is the power set of Ω , serving as the set of all possible **events**. Any subset of Ω is considered an event.
- $\mathbb{P} : 2^\Omega \rightarrow [0, 1]$ is a **probability measure** that assigns a probability to each event in 2^Ω .

To reason quantitatively about random processes on graphs, one also needs the notion of a *random variable*, which maps the result in the sample space to real values.

Definition 27 (Random Variable). *Let $(\Omega, 2^\Omega, \mathbb{P})$ be a finite discrete probability space. A **random variable** is a function $X : \Omega \rightarrow \mathbb{R}$.*

The expected value and the variance are two fundamental characteristics that help us to understand the behavior of random variables. The expected value provides a measure of the tendency or the average-case outcome of a random process, while the variance quantifies the spread or variability of those outcomes. These concepts are essential in analyzing the performance of algorithms, especially when dealing with random input or probabilistic models, as they provide insight into the typical behavior and the degree of fluctuation around the expected outcome.

Definition 28 (Expected Value). Let $(\Omega, 2^\Omega, \mathbb{P})$ be a finite discrete probability space and let $X : \Omega \rightarrow \mathbb{R}$ be a random variable. The **expected value** of X is defined as:

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega) \mathbb{P}(\{\omega\}).$$

If X attains distinct values $x_1, x_2, \dots, x_k$, an equivalent calculation is:

$$\mathbb{E}[X] = \sum_{j=1}^k x_j \cdot \mathbb{P}(X = x_j).$$

Definition 29 (Variance). Let X be a random variable with a finite expected value $\mu = E[X]$. The **variance** of the random variable X , denoted by $\text{Var}(X)$, is defined as:

$$\text{Var}(X) = E \left[(X - \mu)^2 \right].$$

Using these definitions, we can apply a well-known inequality that will be helpful to our analysis. *Markov's inequality* provides a simple tool to bound the probability that a random variable exceeds a certain value.

Theorem 3 (Markov's Inequality). Let $X \geq 0$ be a non-negative random variable, and let $a > 0$. Then:

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

2 Fundamentals of Cut-Path Problem

Having established the foundational tools from graph theory, algorithmics, and probability in the preceding chapter, we now direct our focus to the core subject of this thesis: the **Min Cut-Path Problem**. This chapter serves as a crucial stepping stone, formally defining the Min Cut-Path problem and exploring its fundamental properties. Understanding this problem is not merely a theoretical exercise; it has significant implications in various network applications where exclusive connectivity is paramount.

In this chapter, we will rigorously define the concept of a *cut-path*, which encapsulates this combined notion of a cut and a path. We will explore its formal properties, its relationship to traditional cuts and paths, and introduce a related concept, the *tree-cut*. This foundational understanding will pave the way for analyzing algorithmic approaches and exploring specific instances of the Min Cut-Path problem in subsequent chapters.

2.1 Cut-Path

The notion of a *cut-path* captures a hybrid structure in a graph that combines two fundamental and seemingly opposing elements: a *path* and a *cut*. Intuitively, a *cut-path* between two vertices u and v is a set of edges that simultaneously contains a path connecting u to v , and a cut that, if removed, would disconnect u from v . In this sense, a cut-path ensures both *communication* and *control*.

Definition 30 (Cut-Path). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. A cut-path between u and v is a set of edges $S \subseteq E$ such that:*

- *There exist subsets $A, B \subseteq S$ where:*
 - *A is a cut separating u and v (i.e., removing A disconnects u and v),*
 - *B is a path connecting u and v (i.e., B forms a sequence of edges between u and v).*

Definition 31 (Cut-Paths). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The set of all cut-paths between u and v , denoted by $\text{cut-path}(u, v)$, is defined as:*

$$\text{cut-path}(u, v) = \{S \mid S \text{ is a cut-path between } u \text{ and } v\}.$$

Definition 32 (Value of Minimum Cut-Path). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The minimum size of any cut-path between u and v , denoted by $\text{cp}(u, v)$, is defined as:*

$$\text{cp}(u, v) = \min\{|S| \mid S \in \text{cut-path}(u, v)\}.$$

In this thesis, we will refer to a *min cut-path* between vertices u and v as any cut-path $S \in \text{cut-path}(u, v)$ whose size is equal to the minimum cut-path value $\text{cp}(u, v)$, i.e., $|S| = \text{cp}(u, v)$.

2.2 Tree-Cut

Having formally defined the concept of a cut-path, which combines a cut with a path structure, we now introduce a related concept: the **tree-cut**. The motivation for exploring tree-cuts stems from considering alternative, potentially more robust, ways to combine cuts with spanning subgraphs.

While a path ensures connectivity between two vertices, a **tree**, being acyclic and connected, represents a fundamentally different approach to connecting vertices. Trees are minimal connected structures, efficient in terms of edge usage, and possesses unique properties, especially in the context of planar graphs and graph duality. Unlike paths, trees inherently lack cycles and redundancy in connections.

In the definition of a tree-cut, we replace the *path* component of a cut-path with a *tree* component. This substitution leads to a new type of cut structure with potentially different characteristics and applications, which we will now formally define.

Definition 33 (Tree-Cut). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. A tree-cut between u and v is a set of edges $S \subseteq E$ such that:*

- *There exist subsets $A, B \subseteq S$ where:*
 - *A is a cut separating u and v (i.e., removing A disconnects u and v),*
 - *B is a tree (i.e., B is a subgraph induced by a subset of edges in S , which forms a tree).*

Definition 34 (Tree-Cuts). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The set of all tree-cuts between u and v , denoted by $\text{tree-cut}(u, v)$, is defined as:*

$$\text{tree-cut}(u, v) = \{S \mid S \text{ is a tree-cut between } u \text{ and } v\}.$$

Definition 35 (Value of Minimum Tree-Cut). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. The minimum size of any tree-cut between u and v , denoted by $tc(u, v)$, is defined as:*

$$tc(u, v) = \min\{|S| \mid S \in \text{tree-cut}(u, v)\}.$$

It is important to observe that $tc(u, v) \geq cp(u, v)$, since any tree-cut can be transformed into a cut-path by extracting a u - v path from the tree. This transformation yields a cut-path whose size is at most that of the original tree-cut, implying that the minimum tree-cut value cannot be smaller than the minimum cut-path value. However, it is possible that the actual tree-cut is slightly larger, which does not affect this inequality.

The following two statements demonstrate that the solution to the *min tree-cut* problem shares similar, yet distinct, properties with the *min cut-path* problem. One key distinction is that, in the case of unweighted graphs, the value $tc(u, v)$ can be computed quite easily using the next theorem.

Theorem 4. *Let $G = (V, E)$ be a connected graph and let $u, v \in V$ be two distinct vertices. Then:*

$$tc(u, v) = t(G).$$

Proof. Any tree-cut, by definition containing a tree, is bounded as follows:

$$t(G) \leq tc(u, v).$$

Consider a cut where we take all the edges incident to u . This forms a valid cut, since removing these edges disconnects u from the rest of the graph. For better understanding, we provide an illustrative example:

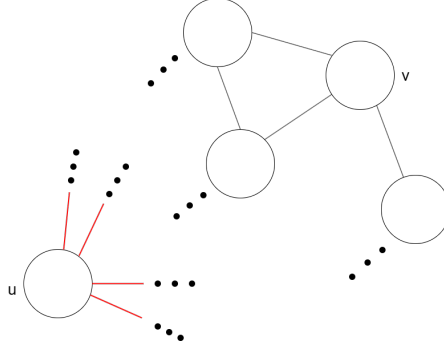


Figure 2.1 Edges Incident to Vertex u Forming a Tree-Cut

These edges do not form a cycle, and by applying Kruskal's algorithm for constructing a minimum spanning tree, we can see that we can always construct a spanning tree.

After constructing this spanning tree, we obtain a valid tree-cut, which implies:

$$t(G) \geq tc(u, v).$$

By combining these two inequalities, we conclude that:

$$t(G) = tc(u, v).$$

□

Theorem 5. For every $n > 2$, there exists a weighted graph $G = (V, E)$ with distinct vertices $u, v \in V$ and $|V| = n$ such that:

$$t(G) \neq tc(u, v).$$

Proof. Consider the complete graph K_n with the vertex set $V = \{v_1, v_2, \dots, v_n\}$, where we set $u = v_1$ and $v = v_n$. The edge weights are assigned as follows:

- The edges forming the path $P_n = (v_1, v_2, \dots, v_n)$, which traverses all vertices, have weight 1.
- All other edges have weight 2.

For better understanding, we provide an illustrative example:

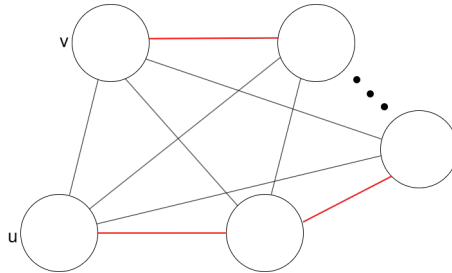


Figure 2.2 Example of Weighted Graph Showing $t(G) \neq tc(u, v)$

Using Kruskal's algorithm, the unique minimum spanning tree consists of the path P_n , as adding any other edge would increase the total weight.

However, this spanning tree does not contain a cut separating u and v , which implies that any valid tree-cut must include additional edges. Therefore, we obtain the following strict inequality:

$$tc(u, v) > t(G).$$

This proves the theorem. $\square$

2.3 Basic Properties of Cut-Path

Having introduced both *cut-path* and *tree-cut*, we now shift our focus back to the original **cut-path problem** and delve deeper into its fundamental properties. Understanding the inherent characteristics and limitations of *cut-paths* is crucial for designing efficient algorithms and analyzing their performance. In this section, we will explore some basic, yet essential, properties of cut-paths, starting with establishing bounds on the size of a minimum cut-path.

Claim 6 (Basic Case). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. If $c(u, v) = 1$ or $d(u, v) = 1$, then:*

$$cp(u, v) = c(u, v) + d(u, v) - 1$$

Proof. Let A be a min cut, so $|A| = c(u, v)$. Let B be a min path, so $|B| = d(u, v)$. Any path between u and v must intersect any cut that separates them, hence $A \cap B \neq \emptyset$.

From this it follows that:

$$cp(u, v) = \max\{c(u, v), d(u, v)\} = c(u, v) + d(u, v) - 1$$

$\square$

Claim 7 (Basic Bounds). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. Then, the value of the min cut-path between u and v satisfies:*

$$\max(c(u, v), d(u, v)) \leq cp(u, v) \leq c(u, v) + d(u, v) - 1,$$

Proof. Proving the upper and lower bounds separately.

1. **Upper bound:** Let A be a minimum cut separating u and v , and let B be a shortest path between u and v . Since A separates u and v and B connects them, taking their union $A \cup B$ forms a cut-path. In the worst case, A and B share exactly one edge, resulting in a total size of $|A| + |B| - 1$. This gives the upper bound:

$$cp(u, v) \leq c(u, v) + d(u, v) - 1.$$

2. **Lower bound:** Any cut-path must contain at least one set forming a u - v cut (of size at least $c(u, v)$) and at least one set forming a u - v path (of length at least $d(u, v)$). Since a *cut-path* must contain both structures, its size must be at least the maximum of these two values:

$$cp(u, v) \geq \max(c(u, v), d(u, v)).$$

This completes the proof. $\square$

Claim 8 (Basic Approximation). *The union of the edges from a solution to the Min Cut Problem and a solution to the Min Path Problem gives a 2-approximation algorithm for the Min Cut-path Problem.*

Proof. Let S be the set of edges obtained by taking the union of a min cut A (with size $c(u, v)$) and a min path B (with length $d(u, v)$). Then, the size of S satisfies:

$$|S| \leq c(u, v) + d(u, v) - 1.$$

Since the optimal cut-path solution satisfies:

$$cp(u, v) \geq \max(c(u, v), d(u, v)),$$

we get:

$$|S| \leq c(u, v) + d(u, v) - 1 \leq OPT + OPT - 1 < 2OPT.$$

Dividing by OPT gives:

$$\frac{|S|}{OPT} < \frac{2OPT}{OPT} = 2.$$

Thus, the algorithm provides a 2-approximation for the minimum cut-path problem. $\square$

One of the key observations regarding the Min Cut-Path problem is that the behavior of cuts and paths significantly differs. The following claim demonstrates a relationship between the min cut and the Min Cut-Path problem; however, no such direct relationship exists between the min path and the Min Cut-Path problem.

Theorem 9 (Global Min Cut-Path). *Let $G = (V, E)$ be a graph. Then the following holds:*

$$\min_{(u,v) \in E} cp(u, v) = \min_{(u,v) \in E} c(u, v).$$

Proof. From Claim 7, we can deduce that the following holds:

$$\min_{(u,v) \in E} c(u, v) \leq c(x, y) \leq cp(x, y), \quad \forall x, y \in V.$$

If we take the minimum cut in the graph and select any edge (x, y) that belongs to this cut, from the upper bound, we know from Claim 7 that:

$$\min_{(u,v) \in E} cp(u, v) \leq cp(x, y) \leq c(x, y) + d(x, y) - 1 = c(x, y).$$

By combining these two inequalities, we obtain:

$$\min_{(u,v) \in E} c(u, v) = \min_{(u,v) \in E} cp(u, v),$$

which is the desired result. $\square$

2.4 Exploiting Partial Solutions

Exploring the properties of cut-path provides valuable theoretical insights into the nature of the problem. However, from an algorithmic perspective, a crucial question is whether we can leverage specific knowledge about the solution structure to design more efficient algorithms. In this section, we address this question by considering scenarios where we possess *partial information* about the minimum cut-path. Specifically, we investigate whether knowing either a path or a cut that is part of a minimum cut-path allows us to find the complete minimum cut-path in polynomial time. This exploration will shed light on potential avenues for algorithm design and problem simplification under certain conditions.

Theorem 10 (Partial Path Property). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. Let P be a path between u and v . If P is a path in a minimum cut-path between u and v , then the minimum cut-path can be found in polynomial time.*

Proof. Let $H = (V, E \setminus P)$, i.e., remove all edges in the path P from G . Compute a minimum cut C in H separating u and v . This can be done using a polynomial-time algorithm for the minimum cut problem, such as Karger's algorithm or the Ford-Fulkerson method. The solution to the minimum cut-path problem is $S = P \cup C$.

1. Feasibility:

The set $S = P \cup C$ is a valid cut-path. By construction:

- P ensures that a path between u and v is preserved.
- C disconnects u and v in H , which means removing C from G (along with P) disconnects u and v .

2. Minimality:

If there existed a cut C' in G such that $|C' \cup P| < |C \cup P|$, we could form a new cut $C'' = C' \setminus P$, which would satisfy $|C''| < |C|$. Since C'' is a valid cut in H , this contradicts the assumption that C is the minimum cut in H .

3. Polynomial-Time Complexity:

- Removing edges in P to construct H takes $O(|E|)$ time.
- Finding a min cut in H using Karger's algorithm runs in $O(n^3)$ time, which is a polynomial.
- Combining P and C takes $O(|P| + |C|)$ time, which is linear in the size of the graph.

Since all steps are polynomial in the input size, the algorithm runs in polynomial time.

□

Theorem 11 (Partial Cut Property). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. Let C be a cut between u and v . If C is a cut in a minimum cut-path between u and v , then the minimum cut-path can be found in polynomial time.*

Proof. Define the auxiliary graph H by contracting all edges of the cut C . This contraction merges the vertex partition containing u into a single supernode s_1 in H , and the partition containing v into a second supernode s_2 . If the contraction of edges in C results in supernodes s_1 and s_2 belonging to different connected components in the auxiliary graph H , then pathfinding algorithms (such as BFS or Dijkstra's) executed on H will correctly determine that there is no path P between s_1 and s_2 in G . Since the construction of the solution requires finding such a path P in H to form the candidate cut-path $S = C \cup P$, the disconnectedness of s_1 and s_2 directly implies that there does not exist a cut-path between u and v .

Compute a shortest path P in H connecting s_1 and s_2 . This can be done using BFS or Dijkstra's algorithm (if weights exist). The solution to the minimum cut-path problem is $S = C \cup P$.

1. Feasibility:

- C is a cut in G .
- P ensures a preserved path between s_1 and s_2 in H , which corresponds to a path between u and v in G that bypasses C .
- Thus, $S = C \cup P$ is a valid cut-path.

2. Minimality:

If there existed a cut P' in G such that $|P' \cup C| < |P \cup C|$, we could form a new path $P'' = P' \setminus C$, which would satisfy $|P''| < |P|$. Since P'' is a valid path in H , this contradicts the assumption that P is the minimum path in H .

3. Polynomial-Time Complexity:

- Contracting edges in C takes $O(|E|^k)$ time.
- Finding the shortest path P in H takes $O(|V| + |E|)$ using BFS.
- Combining C and P takes $O(|C| + |P|)$, which is linear.

All steps are polynomial in input size.

□

3 Min Cut-Path Problem in Special Graph Classes

As established in the preceding chapters, particularly in our exploration of the fundamental properties of the Min Cut-Path problem in Chapter 2, it becomes evident that finding a universally efficient algorithm for general graphs presents a formidable challenge. While classical network flow theory provides polynomial-time solutions for the minimum cut and efficient algorithms exist for finding the shortest paths, the integrated nature of the Min Cut-Path problem significantly elevates its computational complexity. Unlike its constituent subproblems, a universally applicable polynomial-time algorithm for the Min Cut-Path problem in arbitrary graphs currently remains an open question. This inherent complexity necessitates a strategic refinement of our approach. To make progress towards computationally feasible solutions, this chapter adopts a principled strategy: we narrow our focus to specific, structurally constrained classes of graphs. By exploring the Min Cut-Path problem within these special classes, we aim to uncover graph structures where the problem exhibits more tractable behavior, allowing for the design of efficient, and even polynomial-time, algorithms.

This chapter marks a strategic shift towards computational feasibility. To circumvent the inherent computational hardness of the general Min Cut-Path problem, we direct our attention to specific classes of graphs. By imposing structural constraints and focusing on graphs with particular properties, we aim to uncover scenarios where the Min Cut-Path problem becomes more amenable to efficient algorithmic solutions. This approach is not merely about simplification; it is a deliberate exploration of the problem's landscape, seeking out islands of efficiency within a broader sea of complexity. Specifically, this chapter will delve into the analysis of the Min Cut-Path problem in graphs with bounded diameter and graphs with limited cut values. For these specialized graph classes, we will not only analyze the inherent properties of the Min Cut-Path problem, but crucially, we will also develop and present dedicated algorithms designed for their efficient resolution.

3.1 Graphs of Diameter Two

Having established the general complexities of the Min Cut-Path problem, we now turn our attention to specific graph classes where we might uncover more tractable solutions. A natural first step in this direction is to consider a constraint on the maximum distance between vertices. Limiting the shortest path length to a maximum of two edges leads directly to the well-known class of graphs with diameter two, which have been extensively studied in algebraic graph theory [4]. We formally define this class as follows:

Definition 36. *A graph $G = (V, E)$ has diameter k if for every pair of vertices $x, y \in V$, the distance between x and y is at most k . Formally:*

$$\forall x, y \in V : d(x, y) \leq k,$$

where $d(x, y)$ is the distance between the vertices x and y .

Our objective in this section is to analyze the structural properties of graphs with diameter two and to take advantage of these properties to derive key theorems. These theorems will then serve as the foundation for developing a straightforward and efficient algorithm for solving the Min Cut-Path problem within this specific class of graphs. As a first exploration of the structural characteristics of these graphs, we begin by demonstrating a fundamental decomposition property. We will show that, in any graph, a cut C naturally induces a partition of the set of vertices into subsets I, J, K, L , providing a basis for further structural analysis.

Claim 12. *Let $G = (V, E)$ be a graph, and let C be a cut that divides G into two sets A_1 and A_2 (i.e., $V = A_1 \cup A_2$ and $A_1 \cap A_2 = \emptyset$). Then, there exists a unique decomposition of A_1 and A_2 into subsets I, J, K, L such that:*

- $I \cup J = A_1$ $I \cap J = \emptyset$,
- $K \cup L = A_2$, $K \cap L = \emptyset$,
- $\forall u \in J \cup K, \exists v \in V : (u, v) \in C$.
- $\forall u \in I \cup L, \nexists v \in V : (u, v) \in C$.

Proof. We prove the claim explicitly by building the sets I, J, K, L and showing that the decomposition is unique.

1. Construction of the decomposition:

Algorithm 1 Decomposing the vertex set

Require: Graph G , cut C

Ensure: Partition of V into sets I, J, K, L

```

1: for each  $v \in V$  do
2:   if  $v \in A_1$  then
3:     if  $v$  has no edges in  $C$  then
4:       Assign  $v$  to  $I$ 
5:     else
6:       Assign  $v$  to  $J$ 
7:     end if
8:   else if  $v \in A_2$  then
9:     if  $v$  has no edges in  $C$  then
10:      Assign  $v$  to  $L$ 
11:    else
12:      Assign  $v$  to  $K$ 
13:    end if
14:   end if
15: end for
16: Return sets  $I, J, K, L$ 

```

2. Uniqueness of the decomposition:

- The assignment of a vertex v to I, J, K, L depends only on its adjacency to the edges in C and its membership in A_1 or A_2 .

- The decision for v does not affect the assignment of any other vertex u , as the conditions are local to each vertex.
- Therefore, this decomposition is unique and well defined.

□

With the foundational vertex decomposition established for any graph and any cut, we now leverage the specific constraint of diameter two to unveil further structural properties inherent to this class of graphs. Specifically, considering graphs with diameter two, we can demonstrate that the decomposition induced by a cut C is not entirely arbitrary. Instead, the limited distances within the diameter two graphs enforce a more restricted structure on the sets I, J, K, L . The following claim reveals a significant consequence of this diameter constraint:

Claim 13. *Let $G = (V, E)$ be a graph with diameter 2, and let C be a cut that divides G into two sets A_1 and A_2 (i.e., $V = A_1 \cup A_2$ and $A_1 \cap A_2 = \emptyset$). Let I, J, K, L be the decomposition of A_1 and A_2 as described in Claim 12. Then, at least one of the sets I or L is empty. Formally:*

$$I = \emptyset \vee L = \emptyset.$$

Proof. Assume for contradiction that both I and L are non-empty. Let $u \in I$ and $v \in L$ be arbitrary vertices. By the definition of the decomposition:

- Any path from u to v must pass through J and K , as no edges in C connect I to L directly.
- Specifically, any path from u to v must contain at least three edges:
 - One edge from I to K ,
 - One edge from K to J ,
 - One edge from J to L .

Thus, the distance between u and v satisfies:

$$d(u, v) \geq 3.$$

However, this contradicts the assumption that G has diameter 2, since the diameter requires $d(u, v) \leq 2$ for all $u, v \in V$. Therefore, at least one of I or J must be empty. □

Without loss of generality, and for the sake of simplifying the analysis, we assume that $L = \emptyset$.

Now, before focusing again on diameter two graphs, we introduce a claim outlining a key relationship concerning the intersection of the cut and path within any cut-path, a relationship fundamental to our investigation of the Min Cut-Path problem.

Claim 14. *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. Let $c(u, v)$ be a cut that separates u and v , and let $p(u, v)$ be a path that connects u and v . Then, the intersection of $c(u, v)$ and $p(u, v)$ has an odd number of edges. Formally:*

$$|c(u, v) \cap p(u, v)| \equiv 1 \pmod{2}.$$

Proof. Let A_1 and A_2 be the two sets defined by the cut $c(u, v)$, where $u \in A_1$ and $v \in A_2$. Consider the path $p(u, v)$ between u and v . Let $S = c(u, v) \cap p(u, v)$ be the set of edges at the intersection of the cut and the path.

Each edge in S changes the "side" of the cut (i.e., switches between A_1 and A_2) when traversing the path. Without loss of generality, assume that $u \in A_1$ and $v \in A_2$. To move from A_1 to A_2 , the number of times the path crosses the cut must be odd. Specifically:

- If the path crosses the cut an even number of times, the starting and ending sets would be the same (i.e., u and v would both be in A_1 or both in A_2), which contradicts the assumption that $c(u, v)$ separates u and v .
- Therefore, the path must cross the cut an odd number of times, i.e., $|S| \equiv 1 \pmod{2}$.

Thus, the intersection $c(u, v) \cap p(u, v)$ has an odd number of edges. $\square$

Building upon the structural insights gained from the previous claims, particularly Claims 13, and 14, we are now poised to determine a precise formula for the min cut-path value, $cp(u, v)$, in graphs with diameter two. Thanks to the specific structure of graphs with diameter two, and using the established relationships between cuts and paths within the cut-path problem, we can now express $cp(u, v)$ directly in terms of the minimum cut value $c(u, v)$ and the shortest path distance $d(u, v)$. The following theorem formalizes this crucial result:

Theorem 15. *Let $G = (V, E)$ be a connected graph with diameter 2, and let $u, v \in V$ be two distinct vertices. Then, the value of the min cut-path between u and v satisfies the following:*

$$cp(u, v) = c(u, v) + d(u, v) - 1.$$

Proof. We distinguish two cases based on the values of $d(u, v)$ and $c(u, v)$.

1. Case $d(u, v) = 1$ or $c(u, v) = 1$:

By Claim 6, we know states that if $d(u, v) = 1$ or $c(u, v) = 1$, then

$$cp(u, v) = c(u, v) + d(u, v) - 1.$$

2. Case $d(u, v) = 2$ and $c(u, v) \geq 2$

By Claim 7, we have:

$$\max(c(u, v), d(u, v)) \leq cp(u, v) \leq c(u, v) + d(u, v) - 1.$$

We get:

$$c(u, v) \leq cp(u, v) \leq c(u, v) + 2 - 1 = c(u, v) + 1.$$

Assume for contradiction that the equation $cp(u, v) = c(u, v) + d(u, v) - 1 = c(u, v) + 1$ does not hold. Then, the min cut-path problem would have the same size as the min cut, i.e., $cp(u, v) = c(u, v)$. This implies that the path between u and v is a subset of the minimum cut.

By Claim 14, the length of the path in the min cut-path problem must be at least 3. By Claim 13 and w.l.o.g., the graph can be partitioned into three sets I, J, K , where $u \in K$. We observe that $v \in J$. Since there exists a path between u and v in the minimum cut-path, and because this path must lie within the minimum cut, it follows that vertex v cannot belong to the set I because the path is a subset of the cut. It follows that $v \in J$. For better understanding, we provide an illustrative example:

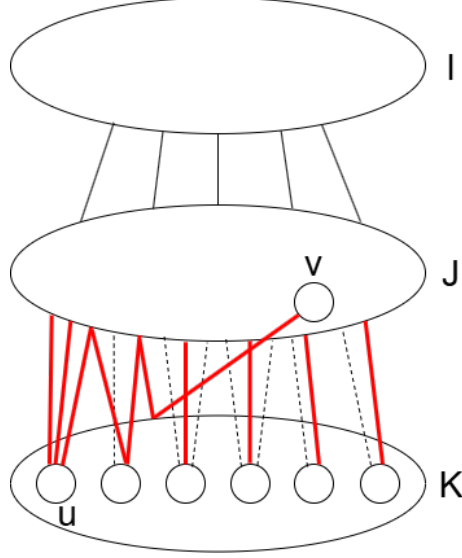


Figure 3.1 Illustration of Structure Cut-Path

We now estimate $cp(u, v)$:

$$cp(u, v) = c(u, v) \geq \deg(u, I) + \deg(u, J) + 2 + (|K| - 2),$$

where the estimation is carried out by decomposing the analysis across the vertices:

- $\deg(u, I)$ and $\deg(u, J)$ denote the number of edges incident from vertex u to the opposite side of the minimum cut,
- from Claim 14 with $d(u, v) = 2$, any u - v path contained in the cut must have length at least three. It follows that there exists at least one vertex other than u from the set K that contributes at least two edges to the cut,
- the remaining $|K| - 2$ vertices from the set K must each contribute at least one edge to the cut.

The right-hand side of the inequality is illustrated in red in Figure 3.1.

Simplifying, we obtain:

$$\deg(u, I) + \deg(u, J) + 2 + (|K| - 2) \geq$$

$$0 + \deg(u, J) + \deg(u, K) + 1 = \deg(u) + 1 \geq c(u, v) + 1.$$

This contradicts the assumption that $cp(u, v) = c(u, v)$, proving that the equation $cp(u, v) = c(u, v) + d(u, v) - 1$ must hold.

□

Having successfully derived a formula for the min cut-path value in diameter two graphs, we now shift our focus to the minimum cut problem itself within the same class of graphs. Remarkably, and as we demonstrate with the following theorem, the minimum cut value $c(u, v)$ in graphs with diameter two can also be determined very efficiently, even more so than the min cut-path value. In fact, as the next theorem reveals, the minimum cut between any two vertices u and v in a diameter two graph is simply determined by the minimum degree of these two vertices. This surprisingly simple result provides a remarkably efficient method for computing minimum cuts in this specific graph class.

Theorem 16. *Let $G = (V, E)$ be a graph with diameter 2, and let $u, v \in V$ be two distinct vertices. Then, the minimum cut between u and v satisfies:*

$$c(u, v) = \min(\deg(u), \deg(v)).$$

Proof. Since G has diameter 2, every pair of vertices is adjacent or connected by a path of length exactly 2. We consider the minimum cut problem between u and v .

By Claim 13 and w.l.o.g., the graph can be partitioned into three sets I, J, K , where $u \in K$. We illustrate this structure:

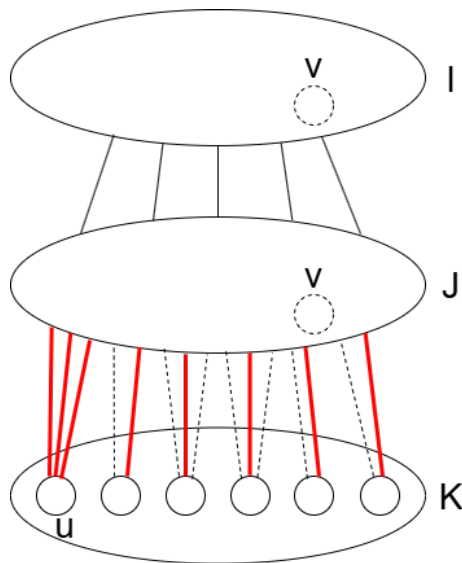


Figure 3.2 Illustration of Structure Cut

We now estimate $c(u, v)$:

$$c(u, v) \geq \deg(u, I) + \deg(u, J) + (|K| - 1),$$

where the estimation is carried out by decomposing the analysis across the vertices:

- $\deg(u, I)$ and $\deg(u, J)$ denote the number of edges incident from vertex u to the opposite side of the minimum cut,

- the remaining $|K| - 1$ vertices from the set K must each contribute at least one edge to the cut.

The right-hand side of the inequality is illustrated in red in Figure 3.2. Simplifying, we obtain:

$$\begin{aligned} \deg(u, I) + \deg(u, J) + (|K| - 1) &\geq \\ \deg(u, I) + \deg(u, J) + \deg(u, K) &= \deg(u). \end{aligned}$$

Thus, we have bounded the cut size by the degree of one of the vertices u or v . However, there exists a trivial cut that separates u (or v) by removing all its incident edges, resulting in a cut of exact size $\deg(u)$ or $\deg(v)$. Since our previous argument provides a lower bound and this trivial cut gives an achievable upper bound, it follows that the minimum cut is attained at a vertex with the smallest degree. Hence, we conclude:

$$c(u, v) = \min(\deg(u), \deg(v)).$$

□

In conclusion, thanks to Theorems 15 and 16, we have achieved a comprehensive understanding of both the min cut-path problem and the minimum cut problem in graphs with diameter two. Specifically, we have shown that for this class of graphs, the min cut-path value, $cp(u, v)$, is directly expressible in terms of the minimum cut value $c(u, v)$ and the shortest path distance $d(u, v)$ using the formula $cp(u, v) = c(u, v) + d(u, v) - 1$. Furthermore, we established that the shortest path distance $d(u, v)$ is trivially determined by checking the presence of an edge between u and v , and that the minimum cut value $c(u, v)$ is efficiently computable as the minimum of the degrees of vertices u and v . These results not only provide valuable theoretical insights into the structure of diameter two graphs but also offer a foundation for efficient algorithmic approaches to the Min Cut-Path problem in this specific, yet important, class of graphs.

3.2 Graphs with Cut-Value at Most Two

Having thoroughly examined the implications of diameter constraints on the Min Cut-Path problem, we now direct our attention to another significant structural property that offers potential for algorithmic simplification. If limiting the diameter of a graph to two allowed us to identify efficient solutions, it is natural to consider the effect of bounding the value of minimum cut between vertices. Therefore, in this section, we explore graphs characterized by a minimum cut-value of at most two for any pair of vertices. This constraint which focuses on the connectivity aspect of the graph, represents another logical progression in our exploration of specialized graph classes, and promises to reveal further avenues for developing efficient algorithms tailored to the Min Cut-Path problem.

Claim 17. *Let $G = (V, E)$ be a connected graph such that for every pair of distinct vertices $x, y \in V$, the minimum cut satisfies:*

$$c(x, y) \leq 2.$$

Then, the graph G can be decomposed into a collection of disjoint cycles or single edges, which are connected by points of articulation.

Proof. We prove the theorem in contradiction. Suppose that the graph G cannot be decomposed into a collection of disjoint cycles or single edges connected by articulation points. Then, there exist two cycles C_1 and C_2 in G that share at least two vertices. It is important to note that these cycles could potentially contain additional edges. However, for the purpose of this proof, it suffices to consider only the edges that form the cycle itself.

We consider a pair of vertices u, v such that both belong to the cycles C_1 and C_2 , and there exists a direct path between them consisting exclusively of the edges of C_2 . If no such path exists, then C_2 must be completely contained within C_1 , which is not possible. After removing this path from the graph, the entire cycle C_1 remains intact, along with a subset of edges from the cycle C_2 . Moreover, the cycle C_1 still contains two additional internally edges-disjoint paths between u and v .

Therefore, any cut that separates u and v must remove at least three edges: one from the cycles C_2 and two from C_1 , which contradicts the assumption that $c(u, v) \leq 2$ for all pairs of distinct vertices $u, v \in V$. Therefore, the graph G must be decomposable into a collection of disjoint cycles or single edges connected by articulation points. $\square$

Given the established tractability of computing both minimum cut values $c(u, v)$ and shortest path distances $d(u, v)$, particularly in the context of specialized graph classes, we now reveal a significant result for graphs where the minimum cut value is bounded by two: for this class, the value $cp(u, v)$ can be directly expressed in terms of $c(u, v)$ and $d(u, v)$, thus paving the way for a polynomial-time determination of $cp(u, v)$.

Theorem 18. *Let $G = (V, E)$ be a connected graph such that for every pair of distinct vertices $x, y \in V$, the minimum cut satisfies:*

$$c(x, y) \leq 2.$$

Then, the value of the min cut-path problem between u and v satisfies:

$$cp(u, v) = c(u, v) + d(u, v) - 1.$$

Proof. We distinguish two cases based on the values of $d(u, v)$ and $c(u, v)$.

1. Case $d(u, v) = 1$ or $c(u, v) = 1$:

In this case we again use Claim 6, which states that if $d(u, v) = 1$ or $c(u, v) = 1$, then:

$$cp(u, v) = c(u, v) + d(u, v) - 1.$$

2. Case $d(u, v) \geq 2$ and $c(u, v) = 2$

From Claim 7, we have the bounds:

$$\max(c(u, v), d(u, v)) \leq cp(u, v) \leq c(u, v) + d(u, v) - 1.$$

Substituting $c(u, v) = 2$, this gives:

$$d(u, v) \leq cp(u, v) \leq d(u, v) + 1.$$

Assume for contradiction that the formula $cp(u, v) = c(u, v) + d(u, v) - 1 = d(u, v) + 1$ does not hold. Given the established bounds, the only remaining possibility is $cp(u, v) = d(u, v)$. This would imply that the minimum cut is contained entirely within a shortest path.

By Claim 17, the graph can be decomposed into a collection $\mathcal{C}$ of disjoint cycles or single edges connected by articulation points. Consider a shortest path P between u and v .

This path cannot traverse any single edge E_i of the collection $\mathcal{C}$, since doing so would result in a cut of size one, contradicts the assumption that $c(u, v) = 2$. Therefore, this path traverses through a sequence of cycles $C_1, C_2, \dots, C_k$. If we remove the set of edges that form P within these cycles, the remaining edges of $C_1, C_2, \dots, C_k$ still provide an alternative path between u and v .

Thus, the edges that form the cut cannot be contained in P . This contradicts the assumption that the min cut-path consists exactly of the minimum path, proving that:

$$cp(u, v) = c(u, v) + d(u, v) - 1.$$

□

3.3 Diam or Cut 2 Graph

In the preceding sections, we explored the Min Cut-Path problem under relatively strict conditions, focusing first on graphs constrained by a maximum diameter of two (Section 3.1) and subsequently on graphs where the minimum cut value between any pair of vertices was at most two (Section 3.2).

These constraints allowed us to derive precise characterizations and efficient solutions. Now, we transition to a broader, more encompassing class of graphs by relaxing these requirements.

In this section, we introduce and analyze graphs that satisfy a combined, less restrictive property: for any pair of vertices, either their distance is at most two or their minimum cut value is at most two. This definition naturally yields a wider family of graphs, potentially sacrificing some of the neat structural properties and straightforward algorithmic efficiency observed in the previous, more specialized cases but allowing us to address a larger set of graph structures.

Definition 37. *Let $G = (V, E)$ be a graph. We say that G has the **diam or cut 2 graph** property if for every pair of distinct vertices $x, y \in V$, at least one of the following conditions satisfies:*

$$c(x, y) \leq 2 \quad \text{or} \quad d(x, y) \leq 2.$$

Unlike the scenarios examined in Sections 3.1 and 3.2, where we successfully established the elegant formula $cp(u, v) = c(u, v) + d(u, v) - 1$ for graphs with a diameter of two and graphs with a maximum cut value of two, respectively, this combined **diam or cut 2** property introduces more complexity.

Relaxation of constraints means that this simple relationship between the minimum cut-path value, the minimum cut value, and the shortest path distance no longer holds universally for all pairs (u, v) within this broader class.

The following figure illustrates specific graph instances within this class that serve as counterexamples, demonstrating scenarios where the aforementioned formula $cp(u, v) \neq d(u, v) + c(u, v) - 1$ fails.



Figure 3.3 Counterexample $cp(u, v) \neq d(u, v) + c(u, v) - 1$

3.3.1 General Square Graph

The counterexamples highlight that a direct application of the formula derived for the more restricted graph classes is insufficient for the **diam or cut 2** category, and we require a more sophisticated approach. This involves constructing a graph structure, which we will term the **general square graph**. This carefully defined structure will allow us to decompose the problem and develop an efficient algorithmic solution tailored to the unique properties of *diam or cut 2* graphs.

Before formally defining the overall *general square graph* structure, we first introduce its fundamental building blocks or pillars. These components, the **general path** and the **square graph**, will be assembled to form the complete decomposition.

Let us start with the **general path**. Informally, think of a general path between vertices u and v as a specific subgraph H connecting these two vertices. Within this subgraph, u and v serve as distinct endpoints, each having only one connection (edge) within subgraph H itself. Importantly, for every edge $\{x, y\} \subseteq V \setminus \{u, v\}$, if one of the vertex belongs to V_H , then the other must also belong to V_H . We will now define this formally:

Definition 38 (General Path). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. A subgraph $H = (V_H, E_H)$ of G is a **general path** between u and v if the following conditions hold:*

1. u and v lie in the same connected component of the subgraph H ,
2. $\deg(u, E_H) = 1, \deg(v, E_H) = 1$,
3. $\{\{x, y\} \in E \mid x \in V_H \setminus \{u, v\} \wedge y \in V\} \subseteq E_H$,

Note that, depending on the structure of the graph G , there may exist several, exactly one, or no general path between a given pair of vertices u and v . One example of a general path is a single edge connecting u and v .

We illustrate a general path as:

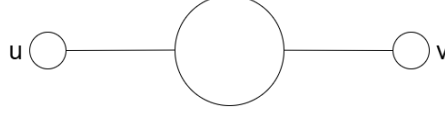


Figure 3.4 General Path

Next, we introduce the **square graph**. Informally, a square graph H is a subgraph associated with four specific vertices a, b, c, d (possibly identical) which must be in V_H . First, the distance between pairs $\{(a, c), (a, d), (b, c), (b, d)\}$ must each be at most 2 in the original graph G . Importantly, for every edge $\{x, y\} \subseteq V \setminus \{a, b, c, d\}$, if one of the vertex belongs to V_H , then the other must also belong to V_H and the vertices a, b, c, d retain all possible edges between them as in the original graph G . Formally:

Definition 39 (Square Graph). *Let $G = (V, E)$ be a graph, and let $a, b, c, d \in V$ be four (possibly identical) vertices. A subgraph $H = (V_H, E_H)$ of G is a **square graph** relative to $\{a, b, c, d\}$ if:*

1. $a, b, c, d \in V_H \subseteq V$,
2. In the graph G , $\forall (x, y) \in \{(a, c), (a, d), (b, c), (b, d)\} : d_G(x, y) \leq 2$,
3. $E_{a,b,c,d} = \{\{x, y\} \in E \mid x \in V_H \wedge y \in \{a, b, c, d\}\}$,
4. $E_{in} = \{\{x, y\} \in E \mid x \in V_H \setminus \{a, b, c, d\} \wedge y \in V\}$,
5. $E_H = E_{a,b,c,d} \cup E_{in}$.

Note that, depending on the structure of the graph G , there may exist multiple distinct square graphs relative to the set $\{a, b, c, d\}$. Moreover, while there may be many reasons for the non-existence of a square graph, it is straightforward to observe that Condition 2 is not satisfied, for instance, when $d(a, c) > 2$.

We illustrate a square graph as:

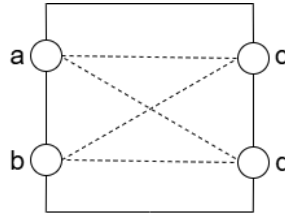


Figure 3.5 Square Graph

We are now ready to define an edge-set decomposition of the original graph G , called the general square graph, as a collection of subgraphs.

Definition 40 (General Square Graph). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. A **general square graph** H between u and v is a collection of subgraphs consisting of square graphs and general paths satisfying the following conditions:*

In order to facilitate a better understanding of these newly introduced concepts, we present an alternative decomposition of the same graph into a general square graph, where the vertices b and c are placed in the interior of the square graph:

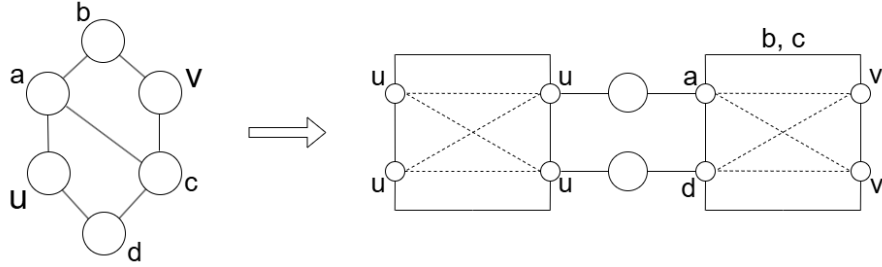


Figure 3.8 Another Example of a Decomposition of The Same Graph into a General Square Graph

This *general square graph* structure, representing a decomposition of the original graph into square graphs and general paths, is the cornerstone of efficient solving of the Min Cut-Path problem within the class of graphs *diam or cut* 2. Its specific construction allows us to develop the polynomial-time algorithms presented in the subsequent sections.

3.3.2 Transformations

Building on the concept of a general square graph, we now turn to its decomposition. Rather than tackling the problem as a whole, we break it down into smaller, more intuitive steps, each designed to simplify the construction process. Throughout this chapter, we will consider the case where $c(u, v) = 2$.

As a first step, we slightly weaken the original concept in order to facilitate a clearer and more transparent understanding of the underlying algorithmic principles.

Specifically, we introduce the notion of a *pseudo-general square graph*, which follows exactly the same structure as a general square graph, except that it consists of *pseudo-square graphs* instead of true square graphs. We now formally define the concept of a pseudo-square graph:

Definition 41 (Pseudo-Square Graph). *Let $G = (V, E)$ be a graph, and let $a, b, c, d \in V$ be four (possibly identical) vertices. A subgraph $H = (V_H, E_H)$ of G is a **pseudo-square graph** relative to $\{a, b, c, d\}$ if:*

1. $a, b, c, d \in V_H \subseteq V$,
2. $E_{a,b,c,d} = \{\{x, y\} \in E \mid x \in V_H \wedge y \in \{a, b, c, d\}\}$,
3. $E_{in} = \{\{x, y\} \in E \mid x \in V_H \setminus \{a, b, c, d\} \wedge y \in V\}$,
4. $E_H = E_{a,b,c,d} \cup E_{in}$.

Note: A *pseudo-square graph* is a square graph without the distance constraint $d(x, y) \leq 2$ between its designated vertices.

We observe that every general square graph is, in particular, a pseudo-general square graph. Consequently, all the results and claims that we establish for

pseudo-general square graphs will remain valid for our final decomposition into a general square graph.

The following claim will assist us in the analysis by showing that the *critical vertices* must lie on edge-disjoint u - v paths.

Claim 19. *Let G be a graph, and let $u, v \in V(G)$ be two distinct vertices. Let $H = \{S_1, P_{1,1}, P_{1,2}, S_2, P_{2,1}, P_{2,2}, \dots, P_{k-1,1}, P_{k-1,2}, S_k\}$ be a pseudo-general square graph decomposition of G between u and v , and let p_1 and p_2 be edge-disjoint u - v paths in G . Then, for every pseudo-square graph S_i of the decomposition, the vertices a_i, b_i, c_i, d_i all lie on the union of the paths $p_1 \cup p_2$.*

Proof. Assume for contradiction that there exists a pseudo-square graph S_i such that one of its designated vertex $x \in \{a_i, b_i, c_i, d_i\}$, does not lie on either path p_1 or p_2 . Since x is not contained in $p_1 \cup p_2$, none of the edges incident to x belong to p_1 or p_2 .

In the structure of a pseudo-general square graph, the transition from S_i to S_{i+1} must occur via the vertices c_i and d_i , through the general paths $P_{i,1}$ and $P_{i,2}$, respectively. From this it follows that both the paths p_1 and p_2 must transition from S_i to S_{i+1} through exactly one of the paths $P_{i,1}$ or $P_{i,2}$. However, by definition, each general path $P_{i,1}$ and $P_{i,2}$ is such that at the vertex of the transition, it has degree one. Therefore, at the point of transition, there is no possibility for two edge-disjoint u - v paths to simultaneously pass through the same general path.

This contradicts the assumption that p_1 and p_2 are edge-disjoint u - v paths in G . Hence, every designated vertex a_i, b_i, c_i, d_i must lie on $p_1 \cup p_2$. $\square$

For algorithmic clarity, I assume further that graph G is connected, although it does not have to be, because disconnected parts simply have no impact on finding the min cut-path. In order to guarantee a complete decomposition into a pseudo-general square graph, all these vertices end up on some general path.

The decomposition process is based on the application of a sequence of transformations. Now, we focus on describing a sequence of transformations that systematically decompose an initial pseudo-general square graph into simpler subgraphs. Each transformation implicitly replaces one current original pseudo-square graph with newly created subgraphs, and the output of each transformation consists of references to the new pseudo-square graphs, which serve as input for subsequent transformations.

It is important to note that, by Menger's Theorem 2 with property $c(u, v) = 2$, there exist two edge-disjoint u - v paths, denoted p_1 and p_2 , which we will fix and frequently refer to in the analysis. Although these paths are not explicitly required within the individual transformations themselves, they play a crucial role in the overall structure of the decomposition.

We assume that S is part of the current decomposition and that the paths p_1 and p_2 pass through S . Furthermore, we assume that the graph G is connected.

Each transformation operates on a given pseudo-square graph S , and is parameterized by two edges of the cut $\{e_1, e_2\}$ and two vertices $x \in \{a, b\}$ and $y \in \{c, d\}$, which are separated in the graph S by the cut $\{e_1, e_2\}$.

Transformation A

Transformation A, depicted in Figure 3.9, is designed to decompose a **pseudo-square** graph S by separating the vertex pairs (a, b) from (c, d) . This separation is achieved through the edges of the cut $\{e_1, e_2\}$, which are in the *pseudo-square* graph. Crucially, these two edges are used to define two disjoint general paths, each containing one of the cut edges. By removing $\{e_1, e_2\}$, the *pseudo-square* graph is naturally partitioned into two subgraphs S_1 and S_2 . These subgraphs inherit all the structural properties required by the definition of a *pseudo-square* graph. The edges e_1 and e_2 form a new general path $P_{1,new}, P_{2,new}$ between S_1 and S_2 .

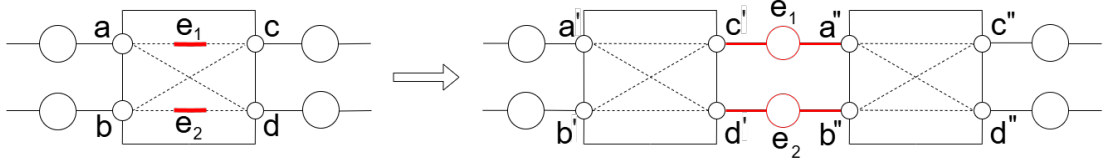


Figure 3.9 Transformation A

Algorithm 2 Transformation A

Require: Graph G , pseudo-square graph S , edges of the cut $e_1, e_2 \in E(S)$, vertices $x \in \{a, b\}, y \in \{c, d\}$

Ensure: Two pseudo-square graphs S_1, S_2

- 1: Initialize $H \leftarrow G$
 - 2: $E(H) \leftarrow E(H) \setminus \{e_1, e_2\}$ ▷ Remove cut edges
 - 3: Initialize $H_x \leftarrow$ x-connected component in H ▷ Find via BFS from x
 - 4: Initialize $H_y \leftarrow$ y-connected component in H ▷ Find via BFS from y
 - 5:
 - 6: Initialize pseudo-square graph $S_1 \leftarrow H_x \cap S$
 - 7: Initialize pseudo-square graph $S_2 \leftarrow H_y \cap S$
 - 8: Initialize general path $P_{new,1} \leftarrow e_1$ ▷ First general path between S_1 and S_2
 - 9: Initialize general path $P_{new,2} \leftarrow e_2$ ▷ Second general path between S_1 and S_2
 - 10: **Return** S_1, S_2
-

Correctness:

We show that replacing S with the subgraphs S_1, S_2 and the two general paths $P_{1,new}, P_{2,new}$ preserves the structure of the pseudo-general square graph.

The transformation is correct because of the following observations. Since the edges cut $\{e_1, e_2\}$ are internal edges of the pseudo-square graph S , the cut induced by their removal is entirely contained within S . Due to Claim 19, the vertices a, b, c and d are located on p_1 and p_2 . The paths p_1 and p_2 ensure that the vertices a, b and c, d remain properly connected through the partial segments of p_1 and p_2 .

Specifically, the partial paths from u to a and b , and from c and d to v , are preserved by removing $\{e_1, e_2\}$. Consequently, the cut defined by $\{e_1, e_2\}$ separates the components containing a and b from those containing c and d , respectively.

The edges $P_{1,\text{new}}$ and $P_{2,\text{new}}$ correspond directly to the single edges e_1 and e_2 , thus trivially satisfying the requirements for general paths in the definition.

Finally, applying BFS starting from the vertices x and y , the components H_x and H_y are correctly identified, ensuring that the resulting subgraphs S_1 and S_2 satisfy all the conditions of a pseudo-square graph.

Transformation B

Transformation B, illustrated in Figure 3.10, addresses scenarios in which one vertex of the set $\{a, b, c, d\}$ becomes isolated from the remaining three vertices due to the cut defined by e_1 and e_2 . This transformation focuses on *extending a general path* by incorporating this newly separated vertex and its associated edges. The edges $\{e_1, e_2\}$ form a new general path P_{new} and a *pseudo-square* graph S_1 .

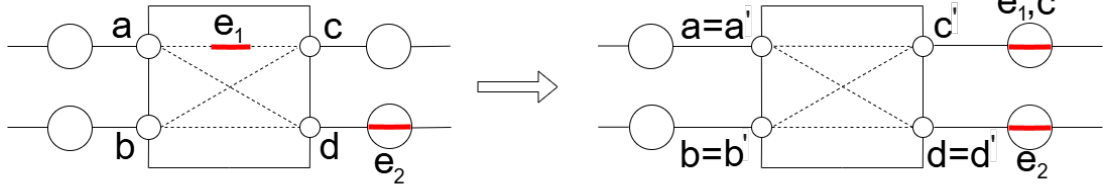


Figure 3.10 Transformation B

Algorithm 3 Transformation B

Require: Graph G , pseudo-square graph S , edges of the cut $e_1 \in E(S), e_2 \notin E(S)$, vertices $x \in \{a, b\}, y \in \{c, d\}$

Ensure: Pseudo-graph S_1

- 1: Initialize $H \leftarrow G$
 - 2: $E(H) \leftarrow E(H) \setminus \{e_1, e_2\}$ ▷ Remove cut edges
 - 3: Initialize $H_x \leftarrow$ x-connected component in H ▷ Find via BFS from x
 - 4: Initialize $H_y \leftarrow$ y-connected component in H ▷ Find via BFS from y
 - 5:
 - 6: $H_{xy} \leftarrow H_x$ or H_y which does not contain any vertex from $\{a, b, c, d\} \setminus \{x, y\}$
 - 7: Find a general path P contain a vertex from H_{xy}
 - 8: Initialize general path $P_{\text{new}} \leftarrow (H_{xy} \cap S) \cup P$ ▷ Extension of a general path
 - 9: Initialize pseudo-square graph $S_1 \leftarrow S \setminus H_{xy}$ ▷ Shrinking of the extended part
 - 10: **Return** S_1
-

Correctness:

We show that replacing S with the reduced pseudo-square graph S_1 together with the extended general path preserves the *pseudo-general* P_{new} .

Similarly to the previous transformation, after the removal of the first edge, which lies inside the pseudo-square graph S , all the vertices a, b, c, d remain connected with the paths p_1 and p_2 due to the Claim 19.

However, by removing the second edge, which lies outside of the pseudo-square graph S , the situation changes: one of the vertices in a, b, c, d becomes isolated from the others. The algorithm then uses BFS to identify which of the vertices has been separated.

The key property is that this isolated vertex can be the property that extended the corresponding general path using the remaining edge of the cut inside S . Thus, the newly formed general path still satisfies the definition required for general paths in the decomposition.

Moreover, the structure of the remaining pseudo-square graph is preserved, except that it becomes slightly smaller.

Transformation C

Finally, **Transformation C**, illustrated in Figure 3.11, addresses the case where both the cut edges e_1 and e_2 lie *outside* the current *pseudo-square graph*. Applying Claim 19, we can conclude that there exist specific pairs of vertices inside the pseudo-square graph that must remain connected. The decomposition correctly identifies these connections and, instead of keeping the original pseudo-square graph, replaces it with the appropriate linking of general paths. The edges $\{e_1, e_2\}$ form new general paths $P_{1,new}$ and $P_{2,new}$.

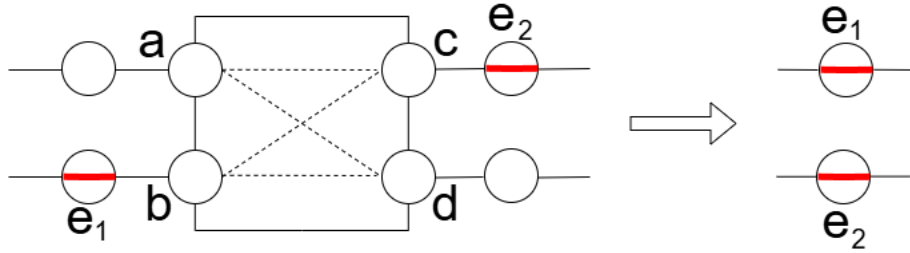


Figure 3.11 Transformation C

Algorithm 4 Transformation C

Require: Graph G , pseudo-square graph S , edges of the cut $e_1, e_2 \notin E(S)$, vertices $x \in \{a, b\}, y \in \{c, d\}$

Ensure:

- 1: Initialize $H \leftarrow G$
 - 2: $E(H) \leftarrow E(H) \setminus \{e_1, e_2\}$ $\triangleright$ Remove cut edges
 - 3: Initialize $H_x \leftarrow$ x-connected component in H $\triangleright$ Find via BFS from x
 - 4: Initialize $H_y \leftarrow$ y-connected component in H $\triangleright$ Find via BFS from y
 - 5:
 - 6: $H_x \leftarrow H_x \cap S$
 - 7: $H_y \leftarrow H_y \cap S$
 - 8: $P_{x,1}, P_{x,2} \leftarrow$ general paths that share a common vertex with H_x
 - 9: $P_{y,1}, P_{y,2} \leftarrow$ general paths that share a common vertex with H_y
 - 10: Initialize general path $P_{1,new} \leftarrow H_x \cap P_{x,1} \cap P_{x,2}$ $\triangleright$ First bridging between the remaining pseudo-square graphs
 - 11: Initialize general path $P_{2,new} \leftarrow H_y \cap P_{y,1} \cap P_{y,2}$ $\triangleright$ Second bridging between the remaining pseudo-square graphs
 - 12: **Return**
-

Correctness:

We show that replacing S with the two general paths $P_{1,new}, P_{2,new}$.

The correctness of the transformation follows from the Claim 19 and the structural properties of the pseudo-square graph S . Since there are no cut-edges inside S , and the paths p_1 and p_2 must traverse through S , they necessarily connect some pairs of vertices from $\{a, b\}$ to $\{c, d\}$.

By performing BFS, the algorithm detects which pairs of vertices are connected, leading to the decomposition of S into two connected components. These components are then used to form two new general paths, corresponding to the segments of the original paths that entered and exited S .

Both newly constructed general paths satisfy all the properties required according to the definition, ensuring that the *pseudo-general square* graph decomposition is correctly preserved after transformation.

Polynomial-Time Complexity of Transformations A, B, and C:

Since the transformations do not contain any cyclic structures and each transformation consists of statements that are $O(1)$, the overall running time can be determined by identifying the most computationally expensive statement among them. In each case, the most time-consuming statement is the application of a BFS algorithm to find the connected components. Since the graph under consideration has at most $O(n^2)$ edges, the time complexity of a single BFS statement is $O(n^2)$. Therefore, each transformation runs in polynomial time.

3.3.3 Polynomial-Time Construction of General Square Graph

Having defined the transformations in the previous subsection, a natural and crucial question arises: can we efficiently construct this decomposition from a given graph that satisfies the *diam or cut 2* property?

This subsection affirmatively answers this question, demonstrating that such a construction is indeed possible in polynomial time.

We begin by describing the core idea behind the algorithm. The algorithm operates first by embedding the entire input graph into a single *pseudo-square* graph structure. This initial embedding trivially satisfies all the conditions of the *pseudo-general square* graph definition, except the number of pseudo-square graphs, which must be at least two by definition.

To address these remaining conditions, the algorithm applies a sequence below described graph transformations A 3.9, B 3.10 or C 3.11 that incrementally decompose the initial graph into smaller subgraphs. Each transformation is designed to reduce the subgraph size while preserving the relevant structural constraints. First of all, we apply Transformation A to ensure that the decomposition yields at least two *pseudo-square* graphs, thus satisfying the first condition.

Subsequently, we iteratively apply transformations to each pseudo-square graph in order to further decompose it into smaller components. This process continues until every resulting pseudo-square graph is sufficiently small to meet the stricter requirements of the square graph.

Algorithm 5 Constructing a General Square Graph Decomposition

Require: Graph $G = (V, E)$ with property diam or cut 2, vertices $u, v \in V$ such that $c(u, v) = 2$

Ensure: A decomposition G into a valid square general graph between u and v .

```

1: Initialize pseudo-square graph  $S_{initial}$  with  $a = u, b = u, c = v, d = v$  and  $V = V(G)$ .
2:  $e_1, e_2 \leftarrow \min cut(u, v)$ 
3:  $S'_1, S'_2 \leftarrow \text{TransformationA}(G, S_{initial}, e_1, e_2, u, v)$ .
4: Initialize  $S \leftarrow \{S'_1, S'_2\}$ .
5: while  $S$  is not empty do
6:    $S_{work} \leftarrow$  select and remove pseudo-square graph from  $S$ .
7:   for each pair  $(x, y)$  in  $\{(a, c), (b, d), (a, d), (b, c)\}$  do
8:     if  $d(x, y) > 2$  then  $\triangleright$  Checking if a pseudo-square graph is a square graph
9:        $e_1, e_2 \leftarrow \min cut(x, y)$   $\triangleright$  Edges  $e_1$  and  $e_2$  are within  $S_{work}$ 
       intesection general paths that connects to this pseudo-square graph.
10:       $Count \leftarrow count(e_1, e_2 \in S_{work})$ 
11:      switch  $Count$  do
12:        case 0
13:           $\text{TransformationC}(G, S_{work}, e_1, e_2, x, y)$ 
14:        case 1
15:           $S'_1 \leftarrow \text{TransformationB}(G, S_{work}, e_1, e_2, x, y)$ 
16:          Add  $S'_1$  to  $S$ .
17:        case 2
18:           $S'_1, S'_2 \leftarrow \text{TransformationA}(G, S_{work}, e_1, e_2, x, y)$ 
19:          Add  $S'_1, S'_2$  to  $S$ .
20:        break
21:      end if
22:    end for
23:  end while
24: Return the constructed general square graph.
```

Intuition of correctness. Each transformation ensures that at least one vertex is incorporated into a general path, after a finite number of iterations, the size of each remaining pseudo-square graph must become sufficiently small. In particular, the distances between any two critical vertices within a pseudo-square graph will eventually be at most two, thereby fulfilling the structural requirements of a general square graph.

However, it is important to emphasize a key prerequisite for the construction presented here. It specifically applies to pairs of vertices (u, v) for which the minimum cut condition holds, i.e., $c(u, v) = 2$. We now present the following proposition along with an algorithm that constructs a general square graph in polynomial time.

Theorem 20. *Let $G = (V, E)$ be a graph satisfying the **diam or cut 2 graph** property. Then, for distinct vertices $u, v \in V$ such that $c(u, v) = 2$, the graph G admits a decomposition into a **general square graph** between u and v .*

Proof. We verify the correctness and polynomial-time complexity of the algorithm.

1. Correctness:

- By Claim 19, we know that in every iteration, all **critical vertices** are contained within the union of the two edge-disjoint u - v paths p_1 and p_2 . Since decomposition requires separating these **critical vertices**, it follows that after removing a single edge from either p_1 or p_2 , the resulting subgraph remains connected. Otherwise, if the removal of a single edge disconnected the graph, the minimum cut between u and v would be of size one, contradicting the assumption that $c(u, v) = 2$.
- Therefore, whenever the algorithm seeks two cut edges e_1 and e_2 in the current pseudo-square graph, it is guaranteed that such a pair exists. Furthermore, by construction, the search for these edges is only invoked when the current pseudo-square graph admits a u - v cut of size at most two. Consequently, we are assured that precisely two such cut edges are found at each step.
- Once the appropriate cut edges e_1 and e_2 are identified, the algorithm applies the corresponding transformation (Transformation A, B, or C) depending on the relative positions of e_1 and e_2 within the structure. Each transformation has been designed to preserve the defining properties of a pseudo-square graph, thus ensuring that the decomposition remains consistent throughout the process.
- The algorithm terminates when every pseudo-square graph becomes sufficiently small to satisfy the strict definition of a square graph. At this point, the entire decomposition forms a valid general square graph, as required.

2. Polynomial-Time Complexity:

- The number of iterations of the *while* loop is bounded by the fact that in each step, either a new pseudo-square graph is created or the number of vertices in the general path increases by 1. Since both the number of pseudo-square graphs and the number of vertices in the general path are bounded by n (the total number of vertices), the maximum number of *while* loop iterations is $2n$.
- In each iteration, we check at most $4n$ (number of pseudo-square graphs), which is bounded by $4n$.
- Finding the minimum cut takes polynomial time (e.g., using an algorithm like the Edmonds-Karp algorithm for maximum flow, which runs in $O(n^3)$), and each transformation takes constant time.
- The complexity of the transformation is $O(n^2)$
- Therefore, the overall complexity is:

$$O(2n) \times O(4n) \times O(n^3 + n^2) = O(n^k),$$

where k is a constant. This proves that the algorithm is polynomial-time.

□

3.3.4 Polynomial-Time Algorithm for Min Cut-Path in Diam or Cut 2 Graph

Now that we have established the existence and polynomial-time constructability of the *general square graph* decomposition for graphs satisfying the *diam or cut 2* property (specifically when $c(u, v) = 2$), we can leverage this structure to devise an efficient algorithm for the Min Cut-Path problem itself.

The core idea is to use the decomposition provided by the general square graph to break down the global problem into a series of localized checks within each constituent *square graph*.

The following claim states that even within the square graphs S_i themselves, we do not always have to find the shortest path.

Claim 21. *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinct vertices. Let $H = \{S_1, P_{1,1}, P_{1,2}, S_2, P_{2,1}, P_{2,2}, \dots, P_{k-1,1}, P_{k-1,2}, S_k\}$ be a decomposition of G into generalized square graphs between u and v . Then, for any shortest $u - v$ path P in G , the following holds:*

$$\forall S_i \in H : |E(S_i) \cap E(P)| \leq 2$$

Proof. Assume for contradiction that there exist a subgraph $S_i \in H$, vertices $(x, y) \in \{(a_i, c_i), (a_i, d_i), (b_i, c_i), (b_i, d_i)\}$, and a shortest path P such that $|E(S_i) \cap E(P)| > 2$.

We distinguish two cases based on the distance $d_{S_i}(x, y)$ in S_i :

1. Case $d_{S_i}(x, y) \leq 2$.

In this case, the path between x and y within S_i has at most two edges. If we replace the segment of P between x and y by the shorter path inside S_i , we obtain a shorter $u-v$ path, contradicting the minimality of P .

2. **Case** $d_{S_i}(x, y) > 2$.

In this case, by the definition of a square graph, any path of length at most two between x and y thus the path must leave S_i and use general paths. Since general paths contain at least one edge, to travel from x to y through external paths, at least two general paths must be used, one to exit S_i and one to reenter it.

Without loss of generality, assume that $x = b_i$ and $y = c_i$ and that the external path exits S_i through b_i . Thus, c_{i-1} is identified with d_{i-1} because I used both general paths. The vertices a_i and c_i are identified ($a_i = c_i$) because we have used all the edges on the general path and cannot continue.

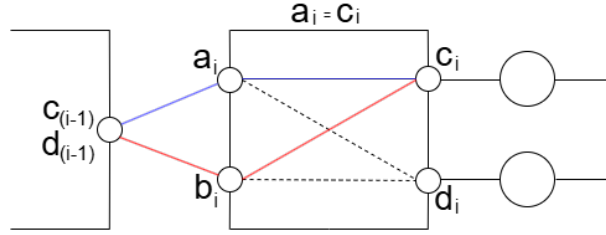


Figure 3.12 Reason Why The Shortest Path Between Any Two Vertices of S_i Must Have Length at Most 2.

Then, as illustrated in Figure 3.12, the path P from d_{i-1} to b_i , and then from b_i to c_i , must use at least four edges. If we replaced this segment by the direct connection between $c_{i-1} = d_{i-1}$ by a direct edge do $b_i = c_i$, we would use only one edge, thus shortening P by at least three edges. This again contradicts the assumption that P is a shortest path.

Therefore, in both cases, we reach a contradiction. Thus, for every $S_i \in H$, we have $|E(S_i) \cap E(P)| \leq 2$. □

Instead of analyzing the entire graph G at once, we focus on examining the paths within each S_i of the decomposition. Within every square graph S_i , we can efficiently determine if there exists a shortest path segment (between two of its defining vertices, say x and y) that simultaneously serves as a cut that separates the other two defining vertices (α and β). If such a path segment exists anywhere along the chain of square graphs and aligns with the overall shortest path between the original vertices u and v , we can conclude that $cp(u, v) = d(u, v)$. Otherwise, $cp(u, v) = d(u, v) + 1$.

This approach of decomposing the problem and performing local checks within the structured components leads directly to the polynomial-time algorithm formalized in the following theorem.

Theorem 22. *Let $G = (V, E)$ be a graph satisfying the property **diam or cut 2 graph**. Then, for every pair of distinct vertices $u, v \in V$ such that $c(u, v) = 2$, the value $cp(u, v)$ can be computed in polynomial time.*

Proof. Based on Theorem 20, we know that for a graph G with the **diam or cut 2** property, there exists a decomposition into a *general square graph*. We utilize

this property to construct a polynomial-time algorithm for computing the value of $cp(u, v)$.

Algorithm 6 Polynomial-time algorithm for computing $cp(u, v)$ in a graph with the **diam or cut 2** property and $c(u, v) = 2$

Require: Graph $G = (V, E)$ with property diam or cut 2, vertices $u, v \in V$ such that $c(u, v) = 2$

Ensure: Value of $cp(u, v)$

```

1: Compute  $d(u, v)$  using breadth-first search
2: Construct the general square graph  $H$  from  $G$  on vertices  $u, v$ 
3:  $S \leftarrow \emptyset$ 
4: for all square graphs  $S_i$  in  $H$  do
5:    $S \leftarrow S \cup \{S_i\}$ 
6: end for
7: for all  $S_i \in S$  do
8:   for all possible vertex pairs  $(x, y)$  as per Definition 39 do
9:      $\{\alpha, \beta\} \leftarrow$  the remaining two vertices of  $S_i$ 
10:     $P \leftarrow$  the set of all shortest paths  $\leq 2$  between  $x$  and  $y$   $\triangleright$  We compute
    the shortest path within the square graph  $S_i$ .
11:    for all  $p \in P$  do
12:      if  $\alpha$  and  $\beta$  are in different components after removing the edges of
      path  $p$  then
13:         $res \leftarrow d(u, x) + d(x, y) + d(y, v)$ 
14:        if  $res = d(u, v)$  then
15:          return  $d(u, v)$ 
16:        end if
17:      end if
18:    end for
19:  end for
20: end for
21: return  $d(u, v) + 1$ 

```

1. Correctness:

According to the Claim 21, for the shortest path segments in the respective S_i , it is sufficient to check up to a maximum length of 2.

We show that the algorithm correctly determines the value of $cp(u, v)$. We know from Claim 7 that:

$$d(u, v) \leq cp(u, v) \leq d(u, v) + 1$$

By Theorem 20, there exists a decomposition of graph G into a *general square graph*. The value $cp(u, v)$ equals $d(u, v)$ if and only if there exists a path that:

- is part of the shortest path between u and v ,
- upon removal, separates the remaining two vertices (α and β) into different components.

If no such path exists in any square graph, an alternative path connecting u and v can be found, ensuring that the cut is not a subset of the shortest path, using connectivity (α and β). The algorithm systematically:

- iterates through all square graphs S_i ,
- examines all vertex pairs (x, y) ,
- evaluates all shortest paths between x and y ,
- checks whether removing any of these paths separates the remaining vertices α and β ,
- if such a path is found, sets $cp(u, v) = d(u, v)$, otherwise, $cp(u, v) = d(u, v) + 1$.

2. Polynomial-Time Complexity:

Each step of the algorithm runs in polynomial time:

- Constructing the *general square graph* takes $O(n^k)$.
- The first **for** loop runs at most $O(n)$ iterations, as each S_i contains at least one vertex of G .
- The second **for** loop has a constant size of 4 (for the four pair vertices in a square graph), i.e., $O(1)$.
- Finding all shortest paths is at worst $O(n^2)$, as the maximum path length is at most 2.
- Checking whether vertices α, β are in the same component takes $O(n^2)$.
- Computing the shortest path takes $O(n^2)$.

Therefore, the overall complexity is:

$$O(n^k + n^7).$$

□

3.3.5 Linear-Time Algorithm for Min Cut-Path with General Square Graph Decomposition

In the preceding subsection (Section 3.3.4), we demonstrated the feasibility of solving the Min Cut-Path problem in polynomial time for graphs possessing the **diam or cut 2** property, contingent upon the $c(u, v) = 2$ condition and leveraging the *general square graph* decomposition.

Although polynomial time is a significant theoretical achievement, we can further refine this approach to achieve an even more efficient solution. Building upon the foundation of the structure of *general square graph* (which we now assume has been precomputed or is given as part of the input, based on the construction in Section 3.3.3), and incorporating dynamic programming techniques to optimize distance calculations between the components of the decomposition, we can achieve a linear-time algorithm with respect to the number of edges in the graph. This enhancement significantly improves the practical applicability for large graph instances. The following theorem details this linear-time result.

Theorem 23. Let $G = (V, E)$ be a graph satisfying the property **diam or cut 2 graph**. Then, for every pair of distinct vertices $u, v \in V$ such that the decomposition H of G between u and v , where H is a general square graph, the value $cp(u, v)$ can be computed in $O(|E|)$ time.

Proof. First, the algorithm solving this problem will be presented, followed by a proof of its correctness and complexity analysis.

Algorithm 7 Linear-time algorithm for computing $cp(u, v)$ in a graph with the **diam or cut 2** property and H as a general square graph

Require: Graph $G = (V, E)$, vertices $u, v \in V$, general square graph H

Ensure: Value of $cp(u, v)$

```

1: Compute  $d(u, v)$  using BFS
2:  $S \leftarrow \emptyset$ 
3: for all square graphs  $S_i$  in  $H$  do
4:    $S \leftarrow S \cup \{S_i\}$ 
5: end for
6: Compute distance tables  $D_1, D_2$  using dynamic programming, where:
    •  $D_1[x]$  stores  $d(u, x)$  for all  $x \in G$ 
    •  $D_2[x]$  stores  $d(x, v)$  for all  $x \in G$ 
7: for all  $S_i \in S$  do
8:   for all possible vertex pairs  $(x, y)$  as per Definition 39 do
9:      $\{\alpha, \beta\} \leftarrow$  the remaining two vertices of  $S_i$ 
10:     $P \leftarrow \emptyset$ 
11:     $P \leftarrow P \cup \text{last edge of path } \alpha \text{ and } x$ 
12:     $P \leftarrow P \cup \text{last edge of path } \alpha \text{ and } y$ 
13:     $P \leftarrow P \cup \text{last edge of path } \beta \text{ and } x$ 
14:     $P \leftarrow P \cup \text{last edge of path } \beta \text{ and } y$ 
15:    if the shortest path distance from  $x$  to  $y$  is  $\leq 1$  then
16:       $P \leftarrow$  the unique shortest path
17:    else
18:      Ensure all partial paths in  $P$  are of length 2; if not, remove edges
        violating this.
19:    end if
20:    for all  $p \in P$  do
21:      if  $\alpha$  and  $\beta$  are in different components after removing the vertices
        of path  $p$  then
22:         $res \leftarrow D_1[x] + d(x, y) + D_2[y]$ 
23:        if  $res = d(u, v)$  then
24:          return  $res$ 
25:        end if
26:      end if
27:    end for
28:  end for
29: end for
30: return  $d(u, v) + 1$ 

```

1. Correctness:

Similar to Theorem 22, the algorithm correctly determines the value of $cp(u, v)$ using the general square graph H . The main difference is the use of dynamic programming, which allows efficient distance calculations.

A crucial part of the correctness proof is the reduction of potential paths between x, y to at most four options. These paths are the only relevant candidates because:

- To disconnect α and β , the removal of either edge (x, z) or (z, y) must be sufficient.
- If there exist multiple ways to reach x or y , removing (x, z) or (z, y) would still leave an alternative connection, keeping α and β in the same component.
- If there exist multiple ways to reach x or y , removing (x, z) or (z, y) would still leave an alternative connection, keeping α and β in the same component.
- Therefore, only four candidate paths need to be checked for their effect on the connectivity between α and β .

If a valid path p is found, the algorithm correctly returns $cp(u, v) = d(u, v)$. Otherwise, by definition, $cp(u, v) = d(u, v) + 1$, which the algorithm correctly outputs as the fallback value.

2. Linear-Time Complexity:

- Computing $d(u, v)$ using BFS takes $O(E)$.
- Constructing the dynamic tables D_1, D_2 for all square graphs takes $O(2E)$.
- Iterating through all square graphs S_i , each containing at most E_i edges. For each S_i :
 - The number of possible pairs (x, y) is always 4.
 - Finding all candidate paths between x, y : $O(4E_i)$.
 - Checking whether α, β remain connected: $O(4E_i)$.
 - Computing res using D_1, D_2 is $O(1)$.

The time complexity of a single iteration is:

$$O(4E_i + 4E_i + 1) = O(E_i)$$

The overall complexity sums over all S_i :

$$O(E + \sum E_i) = O(E)$$

which is linear in the number of edges.

Thus, the algorithm runs in linear time and efficiently computes $cp(u, v)$.

□

In summary, this section significantly expanded our analysis of the Min Cut-Path problem by considering the broader class of **diam or cut 2** graphs. We established that, under the condition $c(u, v) = 2$, it is possible to construct a specialized decomposition, the *general square graph*, in polynomial time. Using this decomposition, we first developed a general polynomial-time algorithm and subsequently refined it to achieve a highly efficient linear-time algorithm $O(|E|)$ for computing $cp(u, v)$.

Although the requirement $c(u, v) = 2$ was necessary for these efficient algorithmic constructions, it represents a reasonable trade-off, allowing us to address a wider range of graphs compared to the strictly diameter-constrained or cut-constrained classes analyzed earlier. This exploration demonstrates the power of identifying and exploiting specific structural properties within graph classes to overcome computational challenges.

4 Min Cut-Path Problem in Random Graphs

In the previous chapters, we examined classes of graphs in which certain optimization problems can be solved efficiently. However, in general, it is not clear whether, for every graph and given problem, there exists a polynomial-time algorithm that provides an optimal solution. Therefore, in this chapter, we focus on analyzing to what extent these problems can be solved efficiently and how well an optimal solution can be approximated on average.

For this purpose, we introduce **Erdős–Rényi random graphs** [5], which allow us to analyze the structural properties of graphs and their impact on the computational complexity of algorithms solving the **Min Cut-Path** problem. Specifically, we will investigate the properties of random graphs in the context of algorithmic classes with nearly polynomial-time complexity, average-case computational complexity, and approximation schemes.

Definition 42 (Erdős–Rényi Random Graph). *Let $n \in \mathbb{N}$ and $p \in [0, 1]$. The Erdős–Rényi random graph, denoted by $G(n, p(n))$, is a graph with n vertices where each edge between two distinct vertices is included independently with probability p . Formally:*

- The vertex set is $V = \{1, 2, \dots, n\}$,
- For each pair of distinct vertices $u, v \in V$, the edge $\{u, v\}$ is included in E with probability p , independently of all other edges.

4.1 Time Complexity Classes for Random Graph Algorithms

In this chapter, we introduce an extended terminology regarding algorithmic complexity, focusing on cases where worst-case analysis does not provide a meaningful characterization of practical performance. Instead of analyzing the most challenging instances of graphs, we shift our attention to the *average-case* complexity [6], which better reflects the behavior of algorithms in typical instances of random graphs.

The **Min Cut-Path** problem remains computationally challenging even in the context of random graphs. However, we can guarantee an algorithm with a runtime that, while slightly more polynomial, is still significantly faster than exponential-time methods. This motivates the introduction of the concept of an *almost polynomial algorithm* [7], which is widely used in modern algorithmic research.

Definition 43 (Almost Polynomial Algorithm). *An algorithm A is called an almost polynomial algorithm if its running time $T_A(n)$ satisfies*

$$T_A(n) \in O\left(n^{O(\log \log n)}\right).$$

This time complexity is slightly more polynomial, but still subexponential.

Definition 44 (Average-Case Algorithm). *An algorithm A is called an average-case algorithm if there exists, for each input size n , a subset of inputs $I_{\text{opt}}(n)$ on which the algorithm produces an optimal solution, while $I(n)$ denotes the set of all inputs of size n , and it holds that*

$$\lim_{n \rightarrow \infty} \frac{|I_{\text{opt}}(n)|}{|I(n)|} = 1.$$

This means that asymptotically the algorithm produces optimal results for almost all inputs.

Definition 45 (Almost Polynomial Average-Case Algorithm). *An algorithm A is called an almost polynomial average-case algorithm if it possesses the following properties:*

1. **Average-Case Algorithm:** *For each input size n , there exists a subset of inputs $I_{\text{opt}}(n) \subseteq I(n)$ on which A produces an optimal solution, and*

$$\lim_{n \rightarrow \infty} \frac{|I_{\text{opt}}(n)|}{|I(n)|} = 1.$$

2. **Almost Polynomial Algorithm:** *The running time $T_A(n)$ of A satisfies*

$$T_A(n) \in O\left(n^{O(\log \log n)}\right).$$

Definition 46 (Average $(1 + \epsilon)$ -Approximation Scheme). *Let $\epsilon > 0$. An algorithm A is called an average $(1 + \epsilon)$ -approximation scheme if it meets the following criteria:*

1. *A is a polynomial-time algorithm, i.e., its running time is $O(P(n))$ for some polynomial $P(n)$.*
2. *For each input size n , there exists a subset of inputs $I_A^{\text{opt}}(n) \subseteq I(n)$ such that for every input $x \in I_A^{\text{opt}}(n)$ the output $S = A(x)$ satisfies*

$$\frac{S}{\text{OPT}(x)} < 1 + \epsilon \quad (\text{for minimization problems})$$

$$\frac{\text{OPT}(x)}{S} < 1 + \epsilon \quad (\text{for maximization problems}),$$

where $\text{OPT}(x)$ denotes the optimal value for the input x .

3. *In addition, the algorithm is an average-case in the sense that:*

$$\lim_{n \rightarrow \infty} \frac{|I_A^{\text{opt}}(n)|}{|I(n)|} = 1.$$

4.2 Diameter of Erdős-Rényi $G(n, 1/c)$ Graphs

As a first step in our analysis, we examine the structural properties of dense random graphs, specifically those with a high edge probability p . A well-known result in the theory of random graphs states that an Erdős-Rényi random graph $G(n, p)$ with $p = \frac{1}{\alpha}$ (where $\alpha > 0$ is a constant) almost surely has diameter 2 in the asymptotic limit [5].

In the context of the **Min Cut-Path** problem, this property is particularly insightful, as it allows us to take advantage of the efficient methods developed in Chapter 3 Section 3.1 to solve the problem in graphs of diameter 2. One of the key results enabling this is Theorem 15, which shows that when the graph has diameter 2, the value of the *min cut-path* between any two vertices u and v can be computed in polynomial time via the simple formula:

$$cp(u, v) = c(u, v) + d(u, v) - 1.$$

This characterization provides an immediate and efficient way to compute the min cut-path in such graphs, making it a powerful tool for average-case analysis in dense random graphs.

Theorem 24. *Let $G(n, p(n))$ be an Erdős-Rényi random graph with n vertices and edge probability $p(n) = \frac{1}{\alpha}$, where $\alpha > 0$ is a constant. Then, the probability that $G(n, p(n))$ has diameter 2 converges to 1 as $n \rightarrow \infty$. Formally:*

$$\lim_{n \rightarrow \infty} \mathbb{P} \left[\text{diam}(G(n, \frac{1}{\alpha})) = 2 \right] = 1.$$

Proof. Let $G(n, p)$ be an Erdős-Rényi random graph with n vertices and edge probability $p = \frac{1}{\alpha}$, where $\alpha > 0$ is a constant.

For any pair of distinct vertices $u, v \in V$:

- The probability that u and v are not directly connected is:

$$\mathbb{P}[(u, v) \notin E] = 1 - p.$$

- The probability that u and v do not share a common neighbor: For each other vertex $w \in V \setminus \{u, v\}$, the probability that w is not a common neighbor is $(1 - p^2)$. Thus:

$$\mathbb{P}[\text{no } w \text{ is a common neighbor}] = (1 - p^2)^{n-2}.$$

- The probability that $d(u, v) > 2$ is:

$$\mathbb{P}[d(u, v) > 2] = (1 - p) \cdot (1 - p^2)^{n-2}.$$

The expected number of such pairs is:

$$\mathbb{E}[\#d(u, v) > 2] = \binom{n}{2} \cdot (1 - p) \cdot (1 - p^2)^{n-2}.$$

We apply:

$$(1 - p^2)^{n-2} \leq e^{-p^2(n-2)}.$$

Thus:

$$\mathbb{E}[\#d(u, v) > 2] \leq \frac{n^2}{2} \cdot (1 - p) \cdot e^{-p^2 n}.$$

Since $p = \frac{1}{\alpha}$ is a constant, the exponential term $e^{-p^2 n}$ decays rapidly to 0, while the polynomial term n^2 grows slowly. Therefore:

$$\lim_{n \rightarrow \infty} \mathbb{E}[\#d(u, v) > 2] = 0.$$

By Markov's inequality:

$$\mathbb{P}[\text{diam}(G(n, p)) > 2] \leq \mathbb{E}[\#d(u, v) > 2].$$

Since the right-hand side converges to 0, we have:

$$\lim_{n \rightarrow \infty} \mathbb{P}[\text{diam}(G(n, p)) > 2] = 0,$$

and thus:

$$\lim_{n \rightarrow \infty} \mathbb{P}[\text{diam}(G(n, p)) = 2] = 1.$$

□

4.3 Properties of Erdős-Rényi Random Graphs

This chapter investigates the structural properties of the sparse Erdős-Rényi random graphs $G(n, p)$ with $p = \frac{\alpha \log n}{n}$ (where $\alpha > 0$ is a constant). The primary focus is on fundamental results concerning the diameter and the minimum cut of such graphs. Although these results are well-established in the literatures [5, 8], this chapter provides a concise overview, highlighting the statements relevant to the analysis of the **Min-Cut Path** problem. To maintain clarity and brevity, only the necessary theorems are stated, with proofs omitted except for one result regarding edge connectivity $c(u, v)$ between arbitrary pairs of vertex.

Theorem 25 (Connectivity). *Let $G(n, p(n))$ be an Erdős-Rényi random graph with $p(n) = \frac{\alpha \log n}{n}$, where $\alpha > 1$. Then:*

$$\lim_{n \rightarrow \infty} \mathbb{P}[G \text{ is } k\text{-edge-connected}] = 1,$$

where $k = \delta(G)$ denotes the minimum degree of G .

Theorem 26 (Diameter). *Let $G(n, p(n))$ be an Erdős-Rényi random graph with $p(n) = \frac{\alpha \log n}{n}$, where $\alpha > 1$. Then:*

$$\lim_{n \rightarrow \infty} \mathbb{P}\left[\text{diameter of } G \leq \frac{\log n}{\log \log n}\right] = 1.$$

Theorem 27 (Largest Component). *Let $G(n, p(n))$ be an Erdős-Rényi random graph with $p(n) = \frac{\alpha \log n}{n}$, where $\alpha > 1$. Then:*

$$\lim_{n \rightarrow \infty} \mathbb{P}\left[\text{largest component of } G \text{ has size } \Theta(n^{\frac{2}{3}})\right] = 1.$$

Theorem 28 (Degree Concentration). *Let $G(n, p(n))$ be an Erdős–Rényi random graph with*

$$p(n) = \frac{\alpha \log n}{n},$$

where $\alpha > 1$ is a fixed constant. Then, for any fixed $\epsilon > 0$, it holds that

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\forall v \in V(G) : (1 - \epsilon)\alpha \log n \leq \deg(v) \leq (1 + \epsilon)\alpha \log n\right) = 1.$$

The preceding results provide structural constraints on connectivity or the diameter of the graph. However, we must make these findings applicable in the context of the Min Cut-Path problem. The following claim establishes bounds on the value of $c(u, v)$:

Claim 29. *Let $G(n, p(n))$ be an Erdős–Rényi random graph with $p(n) = \frac{\alpha \log n}{n}$, where $\alpha > 1$. Consider two randomly chosen vertices $u, v \in V(G)$. For every $\epsilon > 0$, there exists n_0 such that for all $n > n_0$, the edge connectivity between u and v satisfies:*

$$\lim_{n \rightarrow \infty} \mathbb{P}\left((1 - \epsilon)\alpha \log n \leq c(v, v) \leq (1 + \epsilon)\alpha \log n\right) = 1.$$

Proof. From Theorem 25 with Theorem 28, we know that the graph $G(n, p)$ is k -edge-connected with high probability, where $(1 - \epsilon)\alpha \log n \leq k \leq (1 + \epsilon)\alpha \log n$. This implies that for all pairs of vertices u, v , the edge connectivity satisfies:

$$c(u, v) \geq \min_{x, y \in V(G)} c(x, y) \geq \min_{z \in V(G)} \deg(z) \geq (1 - \epsilon)\alpha \log n.$$

Since the edge connectivity is determined by the minimum number of edges that must be removed to disconnect two vertices, we observe that providing a specific cut may not yield the minimum possible value. However, it gives an upper bound. In particular, for the given pair (u, v) , the degree constraint implies

$$c(u, v) \leq \deg(u) \leq (1 + \epsilon)\alpha \log n$$

By combining these two inequalities, we obtain the desired result:

$$(1 - \epsilon)\alpha \log n \leq c(u, v) \leq (1 + \epsilon)\alpha \log n$$

□

4.4 Average-Case Approximation for Min Cut-Path in Erdős–Rényi Graphs

Based on known results on the diameter and connectivity of Erdős–Rényi random graphs, we can estimate the expected deviation from the optimal solution. Specifically, the combination of the minimum cut and shortest path solutions leads to an approximation scheme. Using the diameter bound from Theorem 26 and the connectivity concentration from claim 29, we ensure that the approximation ratio remains within a factor of $(1 + \epsilon)$ in average-case. This guarantees that the proposed approach qualifies as an average $(1 + \epsilon)$ -Approximation Scheme.

Theorem 30. *Let $G(n, p)$ be an Erdős–Rényi random graph with $p \geq \frac{\alpha \log n}{n}$, where $\alpha > 1$. Then the algorithm that combines the solutions for the min cut and the min path problem is an Average $(1 + \epsilon)$ -Approximation Scheme.*

Proof. Since the probability p increases, the length of the shortest path decreases. From Theorem 26, we obtain the bound:

$$d(u, v) \leq \text{diameter of } G \leq \frac{\log n}{\log \log n}.$$

Using claim 29, we can bound the edge connectivity:

$$c(u, v) > (1 - \epsilon')\alpha \log n.$$

From Claim 7, we have:

$$c(u, v) \leq cp(u, v).$$

All conditions are satisfied except for the third one, which we resolve by substituting:

$$\begin{aligned} \frac{S(x)}{\text{OPT}(x)} &\leq \frac{c(u, v) + d(u, v)}{c(u, v)} \leq \frac{(1 - \epsilon')\alpha \log n + \frac{\log n}{\log \log n}}{(1 - \epsilon')\alpha \log n} = \\ &1 + \frac{\frac{\log n}{\log \log n}}{(1 - \epsilon')\alpha \log n} \leq 1 + \epsilon. \end{aligned}$$

Thus, we conclude that the algorithm achieves a Average $(1 + \epsilon)$ -Approximation Scheme. $\square$

4.5 Almost Polynomial Average-Case Approximation Algorithm in Erdős–Rényi Graphs

By demonstrating an efficient solution for dense graphs in Section 4.2, we can propose a slightly less time-efficient algorithm for sparse graphs, which remains significantly more efficient than exponential solutions. This algorithm, analyzed in the average-case, falls into the category of almost polynomial algorithms. Given the properties of the Erdős–Rényi model with $p = \frac{\alpha \log n}{n}$, we can estimate an upper bound on its computational complexity and show that it performs efficiently on almost all inputs. Conversely, in the subcritical regime where $\alpha < 1$, the probability of a path existing between randomly chosen vertices asymptotically approaches zero, making the algorithm inapplicable in such cases.

Theorem 31. *Let $G(n, p)$ be an Erdős–Rényi random graph with $p = \frac{\alpha \log n}{n}$, where $\alpha > 1$. Then there exists an almost polynomial algorithm that serves as an average-case algorithm.*

Proof. We describe an algorithm that we analyze as an almost polynomial-time algorithm, which exhibits behavior characteristic of an average-case algorithm.

Algorithm 8 Path-Cut

Require: Random graph $G(V, E)$, vertices $u, v \in V(G)$

Ensure: Value $cp(u, v)$

```
1:  $c \leftarrow c(u, v)$  ▷ Minimum cut between  $u$  and  $v$ 
2:  $d \leftarrow d(u, v)$  ▷ Shortest path distance between  $u$  and  $v$ 
3:  $cp \leftarrow \infty$ 
4: Paths = all  $u$ - $v$  paths with length  $\leq c + d$ 
5: for each path  $P$  in Paths do
6:   Create graph  $H$ 
7:    $V(H) \leftarrow V(G)$ 
8:    $E(H) \leftarrow E(G) \setminus P$ 
9:    $Cut \leftarrow c(u, v)$  in  $H$  ▷ New minimum cut in modified graph
10:  if  $cp > Cut + |P|$  then
11:     $cp \leftarrow Cut + |P|$ 
12:  end if
13: end for
14: return  $cp$ 
```

1. Correctness:

The algorithm computes $cp(u, v)$ by iterating through all u - v paths P with length $\leq c(u, v) + d(u, v)$. From Theorem 10, it follows that at least one of the paths must be a partial solution to the *cut-path* problem. By Claim 7, the path length in any cut-path satisfies:

$$|P| \leq c(u, v) + d(u, v).$$

Since the algorithm checks all paths up to this upper bound, it necessarily includes the optimal path-cut combination. The minimality follows from the exhaustive search over all valid candidates.

2. Time Complexity:

- Computing $c(u, v)$ (Karger's algorithm) and $d(u, v)$ (BFS) takes $O(n^k)$.
- Path enumeration is bounded by degree constraints. From Theorem 28, we know that in an Erdős-Rényi random graph $G(n, p)$ with $p(n) = \frac{\alpha \log n}{n}$ and $\alpha > 1$:

$$\Delta = \max_{v \in V} \deg(v) \leq (1 + \epsilon) \alpha \log n.$$

The number of paths of length $\leq c + d$ is thus bounded by:

$$\sum_{k=1}^{c+d} \Delta^k \leq (c+d) \cdot \Delta^{c+d} = O\left((1 + \epsilon) \alpha \log n\right)^{O(\alpha \log n)} = O\left(n^{O(\log \log n)}\right).$$

- Each iteration computes a minimum cut in H in $O(n^{k'})$.

Total complexity:

$$O\left(n^k + n^{O(\log \log n)} \cdot n^{k'}\right) = O\left(n^{O(\log \log n)}\right),$$

satisfying the almost polynomial definition.

3. Average-Case Justification:

The algorithm succeeds on almost all inputs, where $\Delta \leq (1 + \epsilon)\alpha \log n$. By Theorem 28, this occurs with probability:

$$\lim_{n \rightarrow \infty} \mathbb{P}(\Delta \leq (1 + \epsilon)\alpha \log n) = 1.$$

Thus, the algorithm is correct and efficient for asymptotically almost all inputs.

□

In the Erdős–Rényi model, when the edge probability is given by $p(n) = \frac{\alpha \log n}{n}$ with $\alpha < 1$, the graph is known to lose connectivity. This property is crucial in the analysis of cut-paths, which are defined when two vertices u and v belong to the same connected component, guaranteeing the existence of a path between them.

Claim 32. *Let $G(n, p)$ be an Erdős–Rényi random graph with $p = \frac{\alpha \log n}{n}$, where $\alpha < 1$. Then for uniformly random vertices $u, v \in V(G)$:*

$$\lim_{n \rightarrow \infty} \mathbb{P}(cp(u, v) \text{ is defined}) = 0.$$

Proof. For $p(n) = \frac{\alpha \log n}{n}$ with $\alpha < 1$, the graph is in the subcritical regime. By Theorem 27:

$$\lim_{n \rightarrow \infty} \mathbb{P}[\text{largest component of } G \text{ has size } \Theta(n^{\frac{2}{3}})] = 1.$$

For uniformly random u, v :

$$\mathbb{P}(u \text{ and } v \text{ are connected}) \leq \mathbb{P}(u \in V) \cdot \mathbb{P}(v \in \text{largest component of } G).$$

Probability v is in the largest component and $u \in V$:

$$\mathbb{P}(u \in V) = 1$$

$$\mathbb{P}(u \in \text{largest component of } G) \leq \Theta(n^{\frac{2}{3}})/n.$$

Thus:

$$\lim_{n \rightarrow \infty} \mathbb{P}(u \text{ and } v \text{ are connected}) \leq \lim_{n \rightarrow \infty} 1 \cdot \Theta(n^{\frac{2}{3}})/n = 0$$

Since u and v are, with high probability, in different components, the probability of the existence of a path between u and v is low. Consequently, it is highly unlikely to find a valid cut-path.

Hence:

$$\lim_{n \rightarrow \infty} \mathbb{P}(cp(u, v) \text{ is defined}) = 0.$$

□

5 Min Cut-Path Problem: The $c(u, v) = d(u, v) = cp(u, v)$ Case

In this section, we analyze a specific setting in which the notions of *cut*, *distance*, and *cut-path* between a pair of vertices coincide, that is, $c(u, v) = d(u, v) = cp(u, v)$. This scenario captures instances in which a single path simultaneously represents both a *minimum cut* and a *shortest path* between the designated vertices.

A *min cut-path* is any cut-path $S \in \text{cut-path}(u, v)$ such that $|S| = cp(u, v)$. That is, a min cut-path has minimal size among all cut-paths from u to v , and is therefore optimal with respect to both distance and cut.

The notion of a *symmetric cut-path graph* serves as a structural condition to identify graph instances in which the quantities $c(u, v)$, $d(u, v)$, and $cp(u, v)$ coincide. That is, it characterizes graphs where every minimum cut-path from u to v simultaneously realizes the minimum cut and the shortest path between the endpoints. In contrast, *maximum independent path graph* serves a more algorithmic purpose: it defines a subgraph in which all feasible solutions must be contained. This key property, established in claim 33, allows us to restrict the analysis of cut-paths to a significantly smaller subgraph. These two notions are therefore essential: the former captures a special symmetry of the instance, while the latter delimits the domain in which optimal solutions can be effectively sought.

Definition 47 (Symmetric Cut-Path Graph). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinguished vertices. The graph G is called a **symmetric cut-path** graph between u and v if the following equality holds:*

$$c(u, v) = d(u, v) = cp(u, v).$$

Definition 48 (Max Independent Path Graph). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinguished vertices. A collection $\mathcal{P} = \{P_1, P_2, \dots, P_k\}$ of pairwise internally edge-disjoint u - v paths in G is called **maximum** if k is as large as possible.*

*We define the **max independent path** graph between u and v , denoted by $G_{\mathcal{P}} = (V_{\mathcal{P}}, E_{\mathcal{P}})$, as the subgraph of G induced by the union of all paths in maximum $\mathcal{P}$, that is,*

$$V_{\mathcal{P}} = \bigcup_{i=1}^k V(P_i) \quad \text{and} \quad E_{\mathcal{P}} = \bigcup_{i=1}^k E(P_i).$$

Note: An **oriented max independent path** graph is a max independent path graph where we assign an orientation to each path $P_i \in \mathcal{P}$ from u to v .

5.1 Fundamentals of $c(u, v) = d(u, v) = cp(u, v)$ Case

Before delving into the design of algorithms to determine whether a graph constitutes a Symmetric Cut-Path Graph for a given pair of vertices u, v , it is

essential to establish several fundamental claims. These preliminary results will provide a clearer perspective and assist in the subsequent analysis of algorithmic approaches.

Claim 33. *Let $G = (V, E)$ be a symmetric cut-path graph between distinct vertices $u, v \in V$. Let $G_{\mathcal{P}} = (V_{\mathcal{P}}, E_{\mathcal{P}})$ be a max independent path graph between u and v in G .*

If $S \subseteq E$ is a min cut-path, then $S \subseteq E_{\mathcal{P}}$.

Proof. Assume, in contradiction, that the statement is false. Then there must exist an edge on the min cut-path S that is not part of the max independent path graph $G_{\mathcal{P}}$. This would imply that the size of the min cut-path within $G_{\mathcal{P}}$ is at most $c(u, v) - 1$. However, with only $c(u, v) - 1$ edges, it is not possible to form a valid cut in $G_{\mathcal{P}}$, meaning that there exists a path between u and v in $G_{\mathcal{P}} \setminus S$. Consequently, the same path must also exist in G , contradicting the assumption that S was a valid cut-path in $G_{\mathcal{P}}$. Thus, our assumption was incorrect, which is proved by the claim. $\square$

The next two claims presented below highlight a key characteristic of this problem setting: the potential existence of multiple distinct minimum cut-paths within graphs where $c(u, v) = d(u, v) = cp(u, v)$.

Claim 34 (2 Min Cut-Path). *There exists a Symmetric Cut-Path Graph $G = (V, E)$ between distinct vertices $u, v \in V$ that contains at least two distinct min cut-paths between vertices u and v .*

Proof. To establish the claim, it suffices to construct an explicit example of a graph that satisfies the stated properties. This example is illustrated in Figure 5.1, which represents a Symmetric Cut-Path Graph.

In the figure, the two highlighted paths between the vertices u and v are both min cut-paths. Furthermore, the five color-edge-disjoint paths demonstrate the existence of sufficiently many independent paths required to satisfy the symmetric cut-path condition $c(u, v) = d(u, v) = cp(u, v)$.

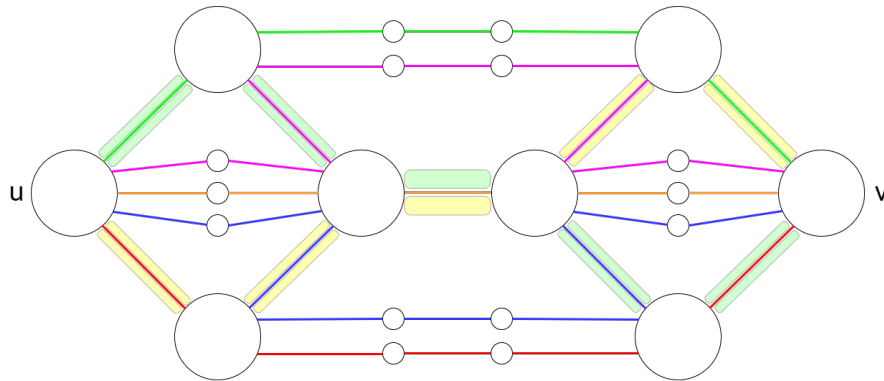


Figure 5.1 Example Graph with Two Distinct Min Cut-Paths

$\square$

The example in Figure 5.1 will be simplified and represented by the diagram below:

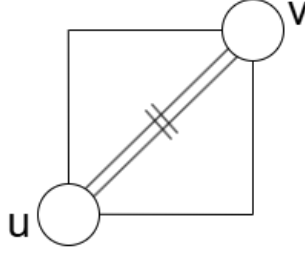


Figure 5.2 Abstract Representation of the Structure from Figure 5.1

For the sake of clarity in the following proof, we consider six edge-disjoint paths of equal length that share the same initial and terminal vertices, as illustrated in the diagram below:



Figure 5.3 Abstract Representation of 6 paths

Claim 35 (Many Min Cut-Paths). *There exists a Symmetric Cut-Path Graph $G = (V, E)$ between distinct vertices $u, v \in V$ such that the number of distinct minimum cut-paths between u and v is $2^{O(\sqrt{n})}$, where $n = |V|$.*

Proof. Observing that multiple locally optimal solutions exist in a small-scale structure (as illustrated in Figure 5.1), we can construct a larger graph by chaining together multiple such substructures in a series, thereby obtaining a graph with exponentially many distinct min cut-paths. This is depicted in Figure 5.4.

To formalize this construction, we define simplified abstract components denoted as blocks $B_1, B_2, \dots, B_k$, where $k = \sqrt{n}$, and each block contains a localized structure similar to the one shown in Figure 5.2. Each block B_i has designated endpoints a_i and b_i , and consecutive blocks are connected so that b_i is adjacent to a_{i+1} for all $i \in \{1, \dots, k-1\}$. The starting and end vertices of the graph are defined as $u = a_1$ and $v = b_k$.

From each endpoint a_i , there exist 6 edge-disjoint paths to u of length $10\sqrt{n}$, and similarly from each b_i to v . This ensures the availability of sufficient connectivity to support a symmetric cut-path structure. For clarity, the simplified representation of a block is illustrated in Figure 5.4:

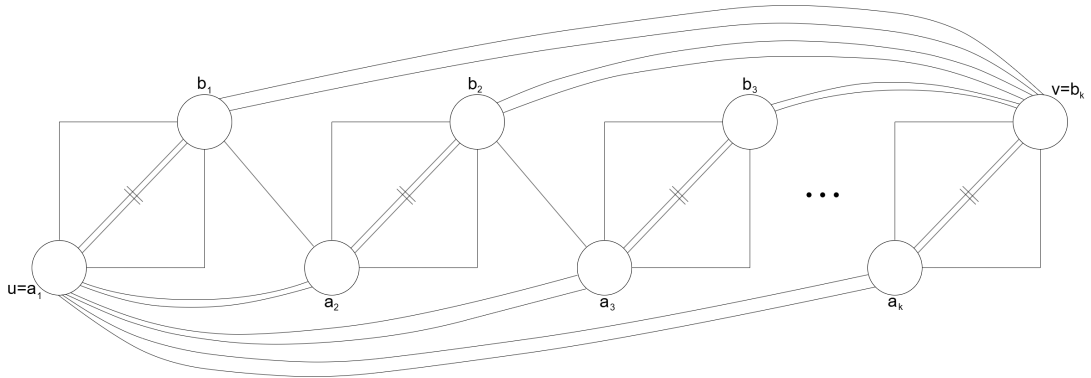


Figure 5.4 Illustration of a Graph with Many Min Cut-Paths

The total number of vertices in the constructed graph is $O(n)$, and any valid min cut-path must traverse all blocks $B_1, \dots, B_k$. Importantly, if a path were to bypass the designated block structure by using edges outside of the blocks or the connections between them, its length would be at least $10\sqrt{n}$, which is strictly greater than the optimal length achievable via the internal block structure. In contrast, using the shortest internal path through each block yields a total path length of only $6\sqrt{n} - 1$.

This implies that every min cut-path must be constructed by selecting, for each block B_i , a path from a_i to b_i that remains entirely within the block. Since each block contains two distinct such paths (by construction), and the blocks are edge-disjoint, the total number of valid min cut-paths is given by $2^{O(\sqrt{n})}$. $\square$

Claim 36 (Local Cut). *Let $G = (V, E)$ be a symmetric cut-path graph between distinct vertices $u, v \in V$, and let $G_{\mathcal{P}} = (V_{\mathcal{P}}, E_{\mathcal{P}})$ be a max independent path graph between u and v in G . Consider any vertex $x \in V_{\mathcal{P}} \setminus \{u, v\}$ and a min cut-path S .*

If $\deg(x, V_{\mathcal{P}}) \leq 4$ and $(w, x) \in S$, then there exists a unique edge $(y, x) \in S$ with $y \neq w$ such that (w, x) and (x, y) are the only edges in S that are incident to x .

Note: This claim implies that even without knowing the full structure of the min cut-path S , the knowledge of a single edge $(w, x) \in S$ helps us infer information about the remaining edges incident to the vertex x .

Proof. We proceed by a case analysis based on the degree of the vertex x within the max independent path graph $G_{\mathcal{P}}$:

1. **Case:** $\deg(x, V_{\mathcal{P}}) = 0$.

Vertex x is not part of any path in $G_{\mathcal{P}}$, hence no edge incident to x can belong to any min cut-path S . The claim holds trivially.

2. **Case:** $\deg(x, V_{\mathcal{P}}) = 1$ or $\deg(x, V_{\mathcal{P}}) = 3$.

These cases are not possible. Every vertex $x \in V_{\mathcal{P}} \setminus \{u, v\}$ lies on a fixed number of edges-disjoint u - v paths. Each path enters and exits x using exactly two edges. Therefore, the degree of such a vertex must be even. Hence, degrees 1 and 3 are excluded.

3. **Case:** $\deg(x, V_{\mathcal{P}}) = 2$.

Let the two edges incident to x in $G_{\mathcal{P}}$ be (x, x_1) and (x, x_2) , each belonging to a different path P_i since S is the min cut-path. Any path between u and v that traverses x must use both edges. However, this would require the cut-path S to contain two edges of the same independent path (since both edges are part of the same path P_i), which contradicts the minimality cut of the cut-path. Therefore, no edge incident to x can belong to a min cut-path S , and the claim holds trivially.

4. **Case 4:** $\deg(x, V_{\mathcal{P}}) = 4$.

In this case, vertex x lies at the intersection of two edge-disjoint paths. For a clearer understanding of the proof, we orient all paths from u to v , transforming $G_{\mathcal{P}}$ into an *oriented max independent path graph* (see Algorithm 12 for how this orientation is later utilized).

As shown in Figure 5.5, there exist precisely two valid configurations of edges incident to x that can form part of a minimum cut-path. These are represented by the blue and purple cuts in the figure. Each edge $(w, x) \in S$ belongs to exactly one of these two configurations, and this uniquely determines the remaining edges incident to x that must also be included in S .

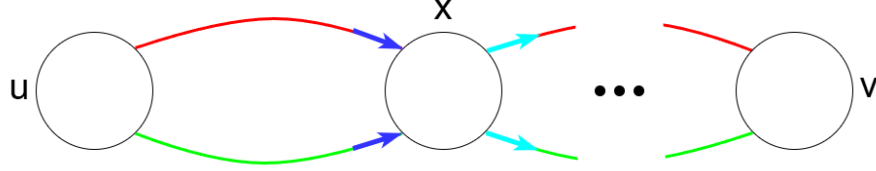


Figure 5.5 Local cut

□

In the proof of Claim 36, we can deduce that in an *oriented Max Independent Path Graph*, any *min cut-path* passing through a vertex x must contain two edges with the same orientation w.r.t. x . This structural insight will be used in the design of Algorithm 12.

5.2 Algorithmic Solutions for Min Cut-Path in $c(u, v) = d(u, v) = cp(u, v)$ Case

Having established the key theoretical foundations in the previous section, we are now ready to turn our attention to algorithmic approaches to solving this particular variant of the problem. In what follows, we present two algorithms that aim to identify minimum cut-paths in the case where the input graph is a symmetric cut-path: the FILTER-BFS algorithm and the LOCAL-CUT algorithm. We will describe how each algorithm works, explain the ideas behind their design, and discuss their strengths and limitations, especially in terms of correctness and computational efficiency. Moreover, if neither algorithm finds a valid solution, this means that the input graph is not a symmetric cut-path.

5.2.1 Filter-BFS Algorithm

A fundamental algorithm for solving shortest path problems in unweighted graphs is the Breadth-First Search (BFS), which systematically explores the graph to identify shortest paths from a given source vertex. The FILTER-BFS algorithm builds on the core principles of BFS but modifies it significantly to address the specific requirements of the *min cut-path* problem in the case where $c(u, v) = d(u, v) = cp(u, v)$.

The goal is to identify paths that are simultaneously the shortest with respect to distance and induce a minimum u - v cut. This adaptation is primarily achieved through the incorporation of two specialized filtering procedures, FILTER-CUT (Algorithm 9) and FILTER-PATH (Algorithm 10), which are applied during the

exploration phase of the route. These filters eliminate partial paths that cannot be extended into valid solutions based on whether they satisfy both the minimum cut and minimum distance conditions.

Algorithm 9 Filter-Cut

Require: Graph $G(V, E)$, list of paths P , vertices $u, v \in V$, cut value c

Ensure: Filtered list of paths P'

```

1:  $P' \leftarrow \emptyset$ 
2: for each path  $p$  in  $P$  do
3:   Create graph  $H$ 
4:    $H(V) \leftarrow G(V)$ 
5:    $H(E) \leftarrow G(E) \setminus p$ 
6:    $cut \leftarrow c(u, v)$  ▷ Minimum cut in graph  $H$ 
7:   if  $c = cut + |p|$  then
8:      $P' \leftarrow P' \cup \{p\}$  ▷ Keep cut if equality holds
9:   end if
10: end for
11: return  $P'$ 

```

Algorithm 10 Filter-Path

Require: Graph $G(V, E)$, list of paths P , vertices $u, v \in V$, path length d

Ensure: Filtered list of paths P'

```

1:  $P' \leftarrow \emptyset$ 
2: for each path  $p$  in  $P$  do
3:   Create graph  $H$ 
4:    $H(V) \leftarrow G(V)$ 
5:    $x \leftarrow p[-1]$  ▷ Last vertex of the path  $p$ 
6:    $path \leftarrow d(x, v)$  ▷ Min path in graph  $G$ 
7:   if  $d = path + |p|$  then
8:      $P' \leftarrow P' \cup \{p\}$  ▷ Keep path if equality holds
9:   end if
10: end for
11: return  $P'$ 

```

During its execution, the main FILTER-BFS algorithm (Algorithm 11) maintains lists $\Pi[z]$ for each vertex z , accumulating candidate paths from the source u to z . As paths are extended edge by edge using a priority queue ordered by path length (mimicking BFS), the filters are invoked. Only paths that satisfy both filter conditions are retained in Π and considered for further extension. Consequently, the final list $\Pi[v]$ contains the identified minimum cut-paths. Although the FILTER-CUT procedure is crucial to ensure that the final paths meet the cut requirement, FILTER-PATH acts primarily as an optimization during the search. It helps to avoid exploring path extensions towards vertices that, based on current path length, could not possibly lead to an overall shortest path to v , effectively steering the search away from branches that are *distancing* themselves from the target vertex v in terms of path length.

Algorithm 11 Filter-BFS Algorithm

Require: Graph $G(V, E)$, vertices $u, v \in V$ **Ensure:** List of valid min cut-paths between u and v

```
1:  $c \leftarrow c(u, v)$ 
2:  $d \leftarrow d(u, v)$ 
3: for all  $v \in V$  do
4:    $\Pi[v] \leftarrow \emptyset$  ▷ Initialize all path sets
5:    $dist[v] \leftarrow \infty$  ▷ Distance estimate
6: end for
7:  $\Pi[u] \leftarrow \{\langle u \rangle\}$  ▷ Initialize trivial path
8:  $dist[u] \leftarrow 0$ 
9:  $Q \leftarrow \text{Queue}()$ 
10:  $Q.\text{enqueue}(u)$ 
11: while  $Q \neq \emptyset$  do
12:    $w \leftarrow Q.\text{dequeue}()$ 
13:   Apply FILTER-CUT( $G, \{\Pi[w]\}, u, v, c$ )  $\rightarrow F_{\text{cut}}$ 
14:   Apply FILTER-PATH( $G, F_{\text{cut}}, u, v, d$ )  $\rightarrow \Pi[w]$ 
15:   for all neighbor  $z$  of  $w$  do
16:     if  $dist[z] = \infty$  then ▷ First visit to  $z$ 
17:        $dist[z] \leftarrow dist[w] + 1$ 
18:        $Q.\text{enqueue}(z)$ 
19:     end if
20:     if  $dist[z] = dist[w] + 1$  then ▷ Maintain only shortest paths
21:       for all  $\pi \in \Pi[w]$  do
22:          $\Pi[z] \leftarrow \Pi[z] \cup \{\pi \oplus z\}$  ▷ Extend each path
23:       end for
24:     end if
25:   end for
26: end while
27: return  $\Pi[v]$ 
```

5.2.2 Local-Cut Algorithm

Compared to the FILTER-BFS algorithm, the LOCAL-CUT algorithm is based on a considerably more straightforward principle. In particular, it explores only the paths contained in the maximal independent path graph. Starting from a vertex u , the algorithm attempts to extend each of its incident edges by exploring candidate min cut-paths in the oriented version of the maximal independent path graph. The key observation, following from Claim 36, is that if the graph $G_{\mathcal{P}}$ has maximum degree four, then at every internal vertex there exists at most one possible candidate for the next edge along a potential min cut-path.

Algorithm 12 Local Cut

Require: Vertices $u, v \in V$, oriented max independent path Graph $G_{\mathcal{P}}$

Ensure: List of potencional min cut-paths in G

```
1: Initialize  $Results \leftarrow \emptyset$ 
2:  $d \leftarrow d(u, v)$  ▷ Min path in  $G_{\mathcal{P}}$ 
3:  $P \leftarrow \emptyset$  ▷ Path list
4: for all neighbor  $z$  of  $u$  do ▷ Initial edges from  $u$ 
5:    $P \leftarrow P \cup \{\langle u, w \rangle\}$ 
6: end for
7: for all paths  $p \in P$  do
8:   for  $k = 1$  to  $d - 1$  do ▷ Extend paths up to length  $d$ 
9:      $(w, x) \leftarrow \text{last-edge}(p)$ 
10:     $o \leftarrow \text{orientation}(w, x)$ 
11:     $candidates \leftarrow \{z \mid (z, x) \in E(G_{\mathcal{P}}), \text{orientation}(x, z) = o, z \neq w\}$ 
12:    if  $|candidates| = 1$  then ▷ The only case where the path can be
      expanded, according to Claim 36
13:       $z \leftarrow \text{the single element in } candidates$ 
14:       $p \leftarrow p \oplus z$ 
15:    else ▷ Path cannot be expanded or degree is greater than 4
16:      break
17:    end if
18:  end for
19:   $Results \leftarrow Results \cup \{p\}$ 
20: end for
21: return  $Results$ 
```

Intuition of correctness. Thanks to Claim 36, once the path leaves the start vertex u , every subsequent vertex x in the oriented max independent path graph has at most one permissible outgoing edge that remains on a min cut-path. In other words, after choosing an initial edge at u , the path can never *branch* again it either continues along that unique extension or terminates. This property bounds the total number of candidate paths to at most $O(n)$ and guarantees that the LOCAL-CUT algorithm only explores each once.

5.2.3 Polynomial Cases: Nearly k -Regular Graphs

Before analyzing the efficiency of the individual algorithms, we introduce a specific class of graphs. Under certain assumptions, the algorithms will demonstrate efficient behavior when applied to graphs from this class.

Definition 49 (Nearly k -Regular Graph). *Let $G = (V, E)$ be a graph, and let $u, v \in V$ be two distinguished vertices. The graph G is called **nearly k -regular** if every vertex in $V \setminus \{u, v\}$ has degree at most k , i.e.,*

$$\forall w \in V \setminus \{u, v\} : \deg(w) \leq k.$$

The following theorem demonstrates that, when restricted to the class of nearly 5-regular graphs, the FILTER-BFS algorithm efficiently enumerates all potential min cut-paths within polynomial time.

Theorem 37. *Let $G = (V, E)$ be a graph with distinguished vertices $u, v \in V$, and suppose that G is nearly 5-regular. Then, the FILTER-BFS algorithm determines in polynomial time whether G is a Symmetric Cut-Path Graph between u and v . If it is, the algorithm also finds all corresponding minimum cut-paths.*

Proof. First, we observe that the given graph has a specific structure. If we construct the *max independent path* graph $G_{\mathcal{P}}$, we obtain an almost 4-regular graph.

1. **Correctness:**

The FILTER-BFS algorithm follows a modified breadth-first search (BFS) approach, enriched with filtering techniques that systematically eliminate incorrect solutions, ensuring the accuracy of the algorithm and that only valid solutions are produced.

Specifically, upon reaching the target vertex v , the candidate paths are subjected to the FILTER-CUT and FILTER-PATH procedures to rigorously verify their minimality as both a *cut* and a *path*, respectively, thus confirming their status as a *min cut-path*.

2. **Polynomial Time Complexity:**

- **Decomposition of Complexity:** The algorithm consists of three key components: BFS traversal, filtering mechanisms, and list size management.
- **Bound on List Sizes:** Outside the max independent path graph $G_{\mathcal{P}}$, list sizes remain zero due to Theorem 33.
- **Bound on Path Expansion:** Since the maximum degree in $V_{\mathcal{P}} \setminus \{u, v\}$ is at most 4, Theorem 36 implies that, except for the terminal vertices u and v , the formation of minimal paths cannot involve branching. The FILTER-CUT procedure enforces this constraint by explicitly filtering out branching possibilities (note that this filter is even stronger, as the theorem requires only a local version of the cut property, not a global one). As a consequence, the only possible branching point is the vertex u , which implies that there are at most n minimal paths. Hence, in every vertex, there can be at most $O(n)$ such paths.
- **Overall Complexity Estimate:**

The total complexity of the algorithm can be bounded in a straightforward manner by observing that from each vertex, we apply two filters to each element in its associated list. Since we can bound the list size per vertex, the total time is at most:

$$\begin{aligned} & \text{max list size per vertex} \times (\text{filter-cut} + \text{filter-path}) \times \text{number of vertices} \\ &= O(n) \times O(n^k + n^{k'}) \times O(n) = O(n^{k''}). \end{aligned}$$

Since all steps execute within polynomial time, the algorithm runs in polynomial time.

□

The following theorem establishes that the LOCAL-CUT algorithm, operating on the oriented max independent path graph, is able to identify $O(n)$ potential candidates for a min cut-path in the original graph, provided that the graph G is symmetric cut-path.

It is worth emphasizing that, if the original graph is nearly 5-regular, the construction of a max independent path graph is nearly 4-regular. In this setting, the number of potential candidates that satisfy the condition $c(u, v) = d(u, v) = cp(u, v)$ remains small, and the verification of each candidate is immediate and computationally trivial.

Theorem 38. *Let $G = (V, E)$ be a graph with distinguished vertices $u, v \in V$. Let $G_{\mathcal{P}} = (V_{\mathcal{P}}, E_{\mathcal{P}})$ be an oriented max independent path graph between u and v in G , and suppose that $G_{\mathcal{P}}$ is nearly 4-regular. Then, the LOCAL-CUT algorithm generates a list of candidate min cut-paths in polynomial time for graph G . The number of candidate min cut-paths generated is at most $O(n)$.*

Proof. We prove the correctness and polynomial-time complexity of the algorithm

1. Correctness:

To argue the correctness of the algorithm, we rely on the structural implication of Theorem 36. The algorithm proceeds by treating each vertex as a potential starting edge of a minimal cut-path, resulting in at most $O(n)$ candidate paths. Due to the local cut constraint guaranteed by the theorem, the expansion of any such path is uniquely determined: for each edge along a candidate path, there exists at most one possible continuation.

Consequently, the algorithm correctly identifies at most $O(n)$ min cut-paths, and none of these paths can branch into multiple options. This guarantees correctness.

2. Polynomial-Time Complexity:

- At most $O(n)$ candidate paths are generated.
- Each path has a length $O(n)$, and extending a path by one edge takes $O(1)$ time.
- **Overall Complexity:**

$$O(n) \times O(n) = O(n^2).$$

Thus, the LOCAL-CUT algorithm terminates in polynomial time.

□

5.2.4 Limitations of Filter-BFS and Local-Cut Algorithms

Although the preceding subsection established the polynomial-time efficiency of the FILTER-BFS and LOCAL-CUT algorithms for specific classes of graphs, namely those that are *nearly 5-regular* (for FILTER-BFS) or a *nearly 4-regular*

oriented *max independent path graph* $G_{\mathcal{P}}$ (for LOCAL-CUT). We now turn our attention to the specific shortcomings of each algorithm.

The FILTER-BFS algorithm guarantees the correctness by systematically exploring the paths using a BFS approach combined with the FILTER-CUT and FILTER-PATH mechanisms. However, its general efficiency is severely hampered by its objective to find **all possible** minimum cut-paths between the designated vertices u and v . As established in Claim 35, the number of distinct minimum cut-paths can be exponential in the size of the graph, potentially reaching $2^{O(\sqrt{n})}$. Consequently, even if the path generation were efficient, the sheer volume of the output can lead to exponential space requirements for storage. This inherent need to potentially enumerate an exponential number of solutions prevents the FILTER-BFS algorithm from achieving polynomial-time complexity in the general case.

The LOCAL-CUT algorithm offers a potentially faster approach by directly tracing paths within the oriented *max independent path graph* $G_{\mathcal{P}}$. Its efficiency is heavily dependent on the structural property suggested by claim 36. Although this holds in well-structured cases such as *nearly 4-regular* graphs, this assumption fails in general graphs. For example, in dense graphs like the complete graph K_n , a vertex x within $G_{\mathcal{P}}$ might have multiple outgoing edges satisfying the orientation criteria. The algorithm's reliance on a unique successor path breaks down in such scenarios, potentially causing it to miss valid minimum cut-paths or even fail to find any solution, thus compromising its correctness for general graphs with high-degree vertices.

Conclusion

This thesis embarked on an exploration of the Min Cut-Path problem, a combinatorial optimization challenge arising from the integration of fundamental graph-theoretic concepts: paths and cuts. We aimed to understand the structural properties and algorithmic complexity inherent in requiring an edge set to simultaneously facilitate connection through a path and enforce separation via a cut between two designated vertices, u and v . The inherent tension between these two objectives makes the Min Cut-Path problem significantly more complex than the well-understood problems of finding shortest paths or minimum cuts independently.

Throughout this work, we have systematically investigated the Min Cut-Path problem across various settings, achieving several key results that contribute to its understanding:

- We established foundational properties and bounds for the Min Cut-Path value $cp(u, v)$, clarifying its relationship, and often its distinction, from $d(u, v)$ and $c(u, v)$.
- A main contribution of this thesis lies in the analysis of Min Cut-Path on special graph classes. While demonstrating tractability for graphs with diameter two and graphs with cut at most two, our most significant result in this area concerns the broader class of *diam or cut 2* graphs. For instances where $c(u, v) = 2$, we introduced the novel concept of *general square graph* decomposition and leveraged it to develop a polynomial-time algorithm. This demonstrates that Min Cut-Path can be solved efficiently even when neither the diameter nor the cut value is universally bounded by two across the entire graph, provided the condition holds for the specific vertex pair or the graph satisfies the combined property.
- We extended our analysis to the average-case behavior using Erdős-Rényi random graphs. An average-case $(1 + \epsilon)$ -approximation scheme was shown, guaranteeing near-optimal solutions on average. For sparse graphs above the connectivity threshold ($p = c \log n / n, c > 1$), we developed an almost polynomial-time average-case algorithm, suggesting that Min Cut-Path is often manageable in typical sparse network scenarios.
- Finally, we characterized the intriguing symmetric case where $c(u, v) = d(u, v) = cp(u, v)$. We provided structural insights and developed polynomial-time algorithms capable of recognizing this property and finding all minimum cut-paths within the specific context of nearly 5-regular graphs.

In summary, this thesis advances the understanding of the Minimum Cut-Path problem by identifying key graph structures and scenarios where efficient solutions are possible. The development of a polynomial-time algorithm for the non-trivial *diameter or cut 2* class, under the specific assumption $c(u, v) = 2$, stands out as a primary contribution, showcasing how structural decomposition can lead to tractability. It should be noted that this work does not resolve the entire *diameter or cut 2* class, and handling the complementary case where $d(u, v) = 2$ remains

an important open direction. The average-case analyses further complement this by indicating efficiency in random graph models.

Despite these advances, several avenues remain open for future research. Determining the precise computational complexity of the general Min Cut-Path problem, potentially proving its NP-hardness, remains the most prominent open question. Developing approximation algorithms with provable performance guarantees for general graphs would also be highly valuable. Furthermore, extending the analysis to other significant graph classes (e.g., planar graphs, bounded treewidth graphs) or exploring weighted and directed variants of the Min Cut-Path problem could yield deeper insights into this complex interplay between connection and separation in networks.

Bibliography

1. DIESTEL, Reinhard. *Graph Theory*. 6th. Heidelberg: Springer, 2023. Graduate Texts in Mathematics, no. 173. ISBN 978-3-662-67631-2.
2. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 4th. Cambridge, MA: MIT Press, 2022. ISBN 978-0-262-53989-5.
3. DIEZ, David M.; BARR, Christopher D.; ÇETINKAYA-RUNDEL, Mine. *OpenIntro Statistics* [<https://www.openintro.org/book/os/>]. 4th. OpenIntro, 2022. ISBN 978-1-943450-07-7.
4. GODSIL, Chris; ROYLE, Gordon. *Algebraic Graph Theory*. Vol. 207. Springer, 2001. Graduate Texts in Mathematics. ISBN 978-0-387-95220-9.
5. BOLLOBÁS, Béla. *Random Graphs*. 2nd. Cambridge: Cambridge University Press, 2001. ISBN 978-0-521-79722-1.
6. ROUGHGARDEN, Tim. *Beyond the Worst-Case Analysis of Algorithms*. Cambridge: Cambridge University Press, 2021. ISBN 978-1-108-87803-4.
7. LANGE, Julian. Properly learning decision trees in almost polynomial time. *arXiv preprint*. 2021. Available from arXiv: 2109.00637.
8. FRIEZE, Alan; KAROŃSKI, Michał. *Introduction to Random Graphs* [online]. Cambridge: Cambridge University Press, 2016 [visited on 2023-11-15]. ISBN 978-1-107-11850-6. Available from: <https://www.math.cmu.edu/~af1p/BOOK.pdf>.

List of Figures

1.1	Spanning Tree	12
1.2	Contraction of edge e	12
2.1	Edges Incident to Vertex u Forming a Tree-Cut	20
2.2	Example of Weighted Graph Showing $t(G) \neq tc(u, v)$	20
3.1	Illustration of Structure Cut-Path	29
3.2	Illustration of Structure Cut	30
3.3	Counterexample $cp(u, v) \neq d(u, v) + c(u, v) - 1$	34
3.4	General Path	35
3.5	Square Graph	35
3.6	General Square Graph	36
3.7	An Example of a Decomposition of a Graph into a General Square Graph	36
3.8	Another Example of a Decomposition of The Same Graph into a General Square Graph	37
3.9	Transformation A	39
3.10	Transformation B	40
3.11	Transformation C	41
3.12	Reason Why The Shortest Path Between Any Two Vertices of S_i Must Have Length at Most 2.	46
5.1	Example Graph with Two Distinct Min Cut-Paths	61
5.2	Abstract Representation of the Structure from Figure 5.1	62
5.3	Abstract Representation of 6 paths	62
5.4	Illustration of a Graph with Many Min Cut-Paths	62
5.5	Local cut	64

List of Tables

This thesis does not contain any tables.

List of Abbreviations

w.r.t.x – with respect to x

w.l.o.g. – without loss of generality

A Attachments

This thesis contains no attachments.